

Aristotle University of Thessaloniki
Faculty of Engineering
Department of Electrical and Computer Engineering
Intelligent Systems and Software Engineering Labgroup

Behavioral Planning for autonomous vehicles on the CARLA Simulator using cameras

Christina Maria Mastralexi

Supervisors: **Andreas Symeonidis**
Professor
Emmanouil Tsardoulias
Post-Doctoral Researcher

September 2022

Behavioral Planning for autonomous vehicles on the CARLA Simulator using cameras

Christina Maria Mastralexi
cmastral@ece.auth.gr

September 2022

Abstract

Behavioral Planning for autonomous vehicles on the CARLA Simulator using cameras

---

Autonomous driving is developing and evolving rapidly in the recent years. Both the scientific community and industry are making significant efforts to develop the necessary sophisticated technology and ultimately to achieve autonomous driving. Autonomous vehicle technology is expected to provide several advantages compared to traditional vehicles, as in the coming years, it will dramatically increase citizens' safety, reduce transportation time and road congestion.

In order to develop such a solution, it is necessary for the autonomous vehicles to have an excellent perception of the surrounding space, be able to adapt to it, and respond instantly to any changes of the surrounding environment. The vehicle must safely navigate on the road and properly respond to static and dynamic obstacles. In addition, it should evaluate decision scenarios and then, according to the circumstances, select the appropriate response. Thus, autonomous vehicles need to be equipped with appropriate state-of-the-art sensors and also with the appropriate control and decision-making system.

This dissertation focuses on the design and development of such an autonomous driving system. The purpose of the system is to safely navigate on the road recognized via optical means. The Computer Vision OpenCV library is extensively used throughout the project for preprocessing the received RGB image, so as to make it suitable for analysis. The perception system uses the programming interface of the CARLA simulator to integrate an RGB camera. For the identification of dynamic obstacles, visual information is analyzed by the deep learning model YOLO (You Only Look Once) while using Optical Character Recognition (OCR) to export information from road traffic signals and a histogram analysis is performed for traffic light color recognition. In the global path construction system, a guided graph of the map is created and the algorithm A* is used to search for the optimal route. Finally, a Lane Keeping Assistance system and a PID controller are used for the navigation control system.

Christina Maria Mastralexi
Electrical and Computer Engineering,
Aristotle University of Thessaloniki, Greece
September 2022

Acknowledgements

I would like to thank my supervisors for their guidance, support, and insightful feedback throughout the development of this thesis. Their expertise and encouragement have been invaluable in shaping the course of this research.

I am also thankful to my friends and family for their support, understanding, and unwavering encouragement throughout this journey.

Contents

1 Introduction **8**
- 1.1 Problem Statement . . . 11
- 1.2 Objectives . . . 11
- 1.3 Chapter Structure . . . 12

2 Literature Review **13**
- 2.1 State-of-the-art autonomous driving systems . . . 13
  - 2.1.1 Developments in the industry . . . 13
  - 2.1.2 Autonomous Driving Simulators . . . 14
- 2.2 Sensor Systems . . . 15
  - 2.2.1 Camera system . . . 15
  - 2.2.2 Radar Sensors . . . 16
  - 2.2.3 LiDAR Sensors . . . 17
- 2.3 Digital Image Processing . . . 17
  - 2.3.1 Image as a two-dimensional signal . . . 17
  - 2.3.2 Image Geometry . . . 18
  - 2.3.3 Processing . . . 18
- 2.4 Machine Learning . . . 19
  - 2.4.1 Introduction . . . 19
  - 2.4.2 Deep Learning . . . 20
  - 2.4.3 Convolutional Neural Networks . . . 20
- 2.5 Computer Vision . . . 24
  - 2.5.1 Introduction . . . 24
  - 2.5.2 Preprocessing . . . 25
  - 2.5.3 Object Detection . . . 26
- 2.6 Algorithms and tools . . . 31
  - 2.6.1 CARLA Simulator . . . 31
  - 2.6.2 OpenCV Library . . . 33
  - 2.6.3 Hough Transform . . . 34
  - 2.6.4 YOLO and Darknet . . . 35
  - 2.6.5 Tesseract OCR . . . 37

3 Methodology **39**
- 3.1 Subsystems . . . 39
  - 3.1.1 Perception subsystem . . . 39
  - 3.1.2 Navigation subsystem . . . 40
  - 3.1.3 Path planning subsystem . . . 40
- 3.2 Perception System . . . 40

3.2.1 Introduction . . . 40
3.2.2 Implementation . . . 40
3.2.3 Lane Detection . . . 41
3.2.4 Object Detection . . . 46
3.2.5 Recognition of speed signals with Tesseract OCR . . . 49
3.2.6 Histogram Analysis for color recognition . . . 50
3.3 Navigation System . . . 51
3.3.1 Navigation using PID Controllers . . . 51
3.3.2 Lane Keeping . . . 54
3.4 Path planning . . . 56
3.4.1 Introduction . . . 56
3.4.2 Implementation . . . 57

4 Experiments and Results **60**
4.1 Introduction . . . 60
4.2 Assumptions . . . 60
4.3 Evaluation Metrics . . . 60
4.4 Experiments and Results . . . 63
4.4.1 CARLA Town Maps . . . 63
4.4.2 Object Detection . . . 68
4.4.3 Optical Character Recognition (OCR) . . . 69
4.4.4 Traffic Light Color Recognition . . . 71
4.4.5 Lane Detection . . . 71
4.4.6 Navigation using PID Controllers . . . 73
4.4.7 Lane Keeping . . . 74

5 Discussion and Conclusions **76**
5.1 Conclusion . . . 76
5.2 Challenges . . . 77
5.3 Future Work . . . 77

List of Figures

1.1 Levels of autonomy [8] . . . 10

2.1 Vision Van by Mercedes-Benz [13] . . . 14
2.2 LGSVL simulator environment [18] . . . 15
2.3 Sensor placement on autonomous vehicles [24] . . . 17
2.4 AI Disciplines in a Venn Diagram. [27] . . . 20
2.5 Convolutional Neural Network Architecture [29] . . . 21
2.6 Convolution Process [29] . . . 22
2.7 Fully Connected layer [29] . . . 23
2.8 Histogram of Oriented Gradients example [40] . . . 27
2.9 AlexNet Architecture [43] . . . 28
2.10 RCNN Architecture [44] . . . 29
2.11 Contrasting YOLO Architecture with SSD [50] . . . 30
2.12 Performance Evaluation of Object Detection Methods [51] . . . 31
2.13 CARLA simulator [52] . . . 31
2.14 OpenCV [53] . . . 33
2.15 Polar Coordinates in Hough Transform [57] . . . 35
2.16 YOLO Conceptual design [29] . . . 36
2.17 Intersection over Union [58] . . . 37

3.1 Lane Detection Algorithm . . . 42
3.2 Convertion to grayscale . . . 43
3.3 Noise reduction with Gaussian filter . . . 44
3.4 Canny Edge Detector . . . 45
3.5 Region of Interest . . . 45
3.6 Lane Detection on CARLA Simulator Image . . . 46
3.7 labelImg Enviroment . . . 48
3.8 Speed signal recognition process . . . 50
3.9 Masks for detecting green and red . . . 50
3.10 Histogram for green, red and orange respectively . . . 51
3.11 Block diagram of a PID controller [66] . . . 52
3.12 Steady state bias of the proportional gain [67] . . . 53
3.13 Heading line in the CARLA simulator environment . . . 55
3.14 Calculation of the steering angle . . . 56
3.15 A* Implementation in Town01 . . . 57
3.16 A* Implementation in Town02 . . . 58
3.17 A* Implementation in Town03 . . . 58

4.1 Town01 Map . . . 63

4.2 Town01 Environment 63
4.3 Town02 Map 64
4.4 Town02 Environment 64
4.5 Town03 Map 65
4.6 Town03 Environment 65
4.7 Town04 Map 66
4.8 Town04 Environment 66
4.9 Town05 Map 67
4.10 Town05 Environment 67
4.11 Object detection on images from CARLA Simulator 70
4.12 Lane Detection in WetCloudyNoon conditions 72
4.13 Lane Detection in MidRainyNoon condition 72
4.14 Lane Detection in ClearNoon conditions 72
4.15 Junction detection 73
4.16 Navigation using waypoints 74

List of Tables

3.1 Model Parameters 49

4.1 Definition of TP, TN, FP, FN 61
4.2 Definition of the metrics 61
4.3 Images in training and test set 68
4.4 Labeled and Unlabeled images 68
4.5 Model metrics during training 69
4.6 The mAP and IoU performance metrics during training 69
4.7 Performance of the OCR 71
4.8 Performance of the color recognition method 71
4.9 Performance of the junction detection method 73
4.10 Percentage of path execution with PID controller 74
4.11 Percentage of path execution with the Lane Keeping system 75

Acronyms and Abbreviations

ADS Automated Driving Systems

NN Artificial Neural Network

CNN Convolutional Neural Network

DNN Deep Neural Network

ROI Region Of Interest

GPS Global Positioning System

ReLU Rectified Linear Unit

OCR Optical Character Recognition

Chapter 1

Introduction

The rapid development of technology has affected all areas of everyday life, extending notably to the transport sector — an indispensable and intrinsic component of any well-operating society. The pursuit of better, faster and safer transportation solutions makes a decisive contribution to the advancement of societies, a necessity that remains self-evident. With the growth of the world's population, the demand for travel is increasing rapidly, and great progress is being made in fortifying road infrastructure and large-scale vehicle manufacturing, reducing costs and enabling affordable vehicles, thus improving the quality of life. In contrast, these rapid changes have created challenges of a different nature. The impact on the environment and the depletion of fossil fuels are some of the most important, but above all is the escalating frequency of vehicular accidents, leading to human casualties and injuries. Consequently, the pressing need for a more streamlined and secure transportation framework becomes evident.

Changes in the way of life and the rapid development of technology have brought about one of the most remarkable innovations of the 21st century - the autonomous vehicle. The advent of autonomous vehicles is a solution to overcome one last and important problem to optimize the transport process, which is the human element. Human beings operate and guide vehicles through the power of ingrained habits and the intuitive capabilities of the human mind, which adeptly analyzes intricate scenarios and adapts actions accordingly. Detecting lanes on the roadway, interpreting traffic signals, the alternation of day and night are things that every driver can deal with relative ease. Although this ability exists, the majority of road accidents occurring on the roads are caused by human actions, indicating instances of driver inattention or incorrect vehicle handling. An autonomous vehicle operates without a driver, which makes the driving process more efficient and safer. By eliminating the human factor and automating driving, these vehicles markedly elevate the quality and efficiency of the driving experience. These systems are capable of constantly monitoring the road conditions, surrounding vehicles and pedestrians and prepared at all times to make the precise decisions needed to ensure the safe passage of passengers to their intended destinations.

Car manufacturers are competing for supremacy in autonomous vehicle technology. Yet, it remains evident that extant driver support systems face several hurdles before attaining the capability to replicate human driving prowess. Employing

machine learning algorithms emerges as a viable avenue to imbue computers with human-like driving behavior. This in turn has its own challenges, particularly in the realm of data requirements for training. Accomplishing the emulation and generalization of human driving behavior demands an extensive amount of data garnered from human drivers, capturing diverse scenarios and their corresponding responses. In addition to the purely technological challenges, autonomous driving has also triggered a social debate that is still in full swing.

Automating the operation of a vehicle involves the coordination and programming of several independent functions. The more functions of a vehicle are automated and fully independent of the driver, the greater the degree of autonomy of the vehicle. According to the National Highway Traffic Safety Administration (NHTSA) [1], the Society of Automotive Engineers (SAE) [2] and other organisations, such as the German Association of the Automotive Industry (VDA) [3] and Germany Federal Highway Research Institute [4] in Germany, a classification technique has been developed since January 2014 that describes which functions the vehicle performs and which the driver must perform. These guidelines classify vehicle autonomy from level 0 to level 5. Today, most car manufacturers produce cars with Level 2 and Level 3 technologies. This classification includes directional and speed setting actions [5] [6] [7]. These levels are:

- **Level 0: No automation**

  Level 0 covers a wide range of older vehicles. At this level the vehicle possesses no automated driving features and is fully controlled by the driver. The vehicle is equipped with a system which provides momentary driving assistance, such as warning signals or emergency safety actions. The driver therefore must drive the vehicle and monitor any possible warnings or safety activities

- **Level 1: Function-specific automation**

  At this level, control of the vehicle is again in the hands of the driver and is the lowest level of autonomy. It includes the automation of specific functions such as cruise control, lane guidance and automated parallel parking. These functions are intended to facilitate the driver.

- **Level 2: Combined autonomous functions**

  At this level the driver remains in control, but the vehicle can operate in a combined mode. The vehicle can control both steering and accelerating/decelerating under specific conditions. Drivers are responsible for checking the traffic conditions and can take control of the car at any time. The responsibility for the vehicle remains with the driver and the above functions assist the driver.

- **Level 3: Conditional automation**

  Vehicles at this level of automation are equipped with an Advanced Driver System (ADS), which can perform all the functions and procedures necessary to drive the vehicle autonomously from a starting point to a destination. The vehicle can respond to dynamic events in the environment and can fully execute tasks such as changing lanes or accelerating to overtake a slow moving vehicle.

The driver must remain alert and ready to take control if the system is unable to execute the task.

- **Level 4: High automation**

  This level is very similar to the previous level. At this level,there is no need for human intervention in driving. This includes motorway driving at any speed within limits, as well as city driving where the roads are much more complex. Vehicles can perform all driving functions and monitor traffic conditions, but in predefined routes. Despite the vehicle's ability to navigate autonomously, the driver must be able to take control of the vehicle at any time he or she wishes.

- **Level 5: Full Automation**

  At this level, the vehicle's system (ADS) is able to perform all driving functions and monitor the traffic conditions of the environment in which it is operating under any circumstances, completely replacing the driver. Vehicles operating at this level are not restricted to a specific geographical area and can operate fully autonomously in any scenario. The driver of the vehicle is not required to have any assistance in the driving process beyond the definition of the desired destination.

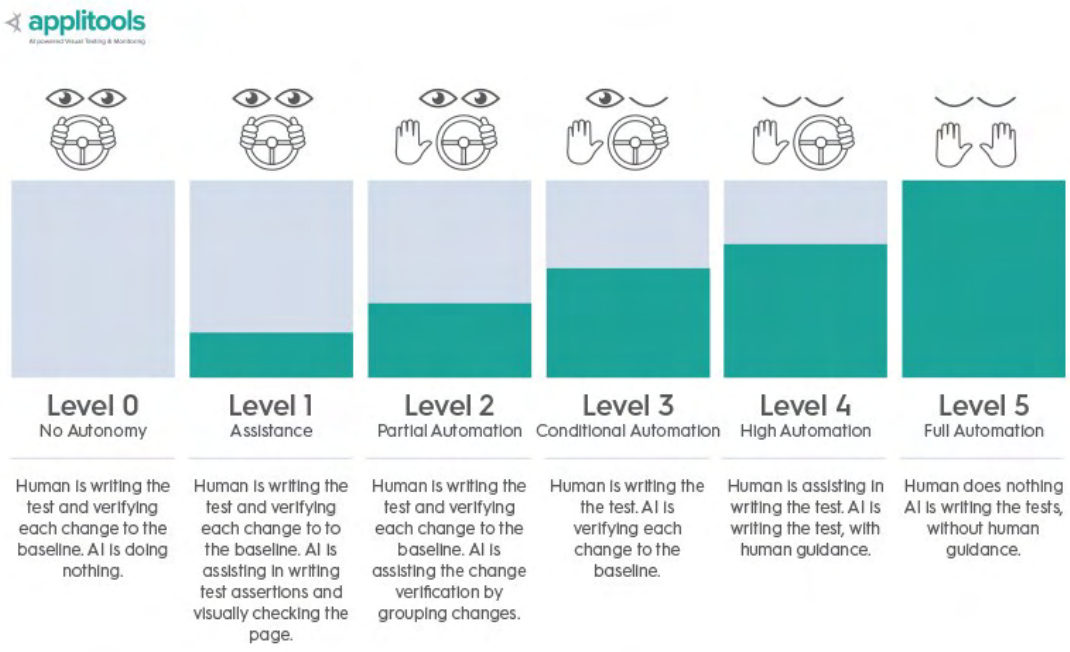


Figure 1.1: Levels of autonomy [8]

Figure 1.1 shows a categorisation of the levels of automation described previously. Autonomous vehicles, as can be understood from the levels of autonomy presented, can offer innovative solutions and opportunities in the field of driving. There is no doubt that the benefits of autonomous driving can be enormous. However, there is also the opposite view, which argues that the costs of acquisition and

operation may be higher than the assumptions of the optimistic forecasts [9] [10] [11]. Fully autonomous vehicles are expected to be technologically developed within this decade, but it will take some years to adapt them to the transport infrastructure and society.

1.1 Problem Statement

The field of autonomous driving has been studied extensively in recent years, yielding techniques that adeptly address numerous challenges intrinsic to these processes. However, the problem of camera-based visual perception of the environment by autonomous vehicles in an unconstrained, dynamic and structured environment with traffic rules, such as the road network, is still a problem with a complex solution.

In a road network, self-driving vehicles must detect other vehicles, pedestrians, recognise road markings while obeying traffic rules, often at high speeds, making the process even more difficult. The volume of data that the vehicle has to process in real time, increases rapidly with the increase in vehicle speed and the processing time.

In the literature there is a plethora of methods that partially describe methodologies for vehicle navigation, control and decision-making. However, there is no proven architecture or model that defines how such a system should be built. The issue remains a focal point of research, driving the continuous development of specialized techniques aimed for the development of the perception and navigation subsystems and their integration.

1.2 Objectives

This thesis aims to introduce an autonomous driving system centered around the camera as the primary sensor, encompassing three essential sub-processes. The vehicle should be able to detect the lanes on the road, detect obstacles within the navigation environment and interpret traffic signals and traffic lights. In addition, it should be able to maintain its position between lanes and navigate from a starting point to a predetermined destination on a known map. The proposed system has three key subsystems:

- **Perception System:** This subsystem leverages camera images to perceive both dynamic and static obstacles within the vehicle's environment. It interprets this data to predict obstacle types, recognize traffic light colors, identify road signals, detect lanes and intersections.
- **Navigation System:** Using information gleaned from the perception system regarding its position on the road and route map, this subsystem enables the vehicle to navigate. It maintains control within lane boundaries and facilitates navigation to the desired endpoint by employing lateral and longitudinal vehicle control inputs.
- **Path Planning System:** This subsystem, commencing from the starting point, generates an optimal basic path on the map leading to the final destination.

The thesis contribution focuses on investigating technologies for the individual subsystems, with the overarching goal of developing a highly functional autonomous driving system.

1.3 Chapter Structure

In this chapter, an introduction was provided, outlining the motivation, problem statement, and objectives of this dissertation. The dissertation has been structured into four additional chapters, each serving a specific purpose:

- **Chapter 2** A review of the research area concerning the current autonomous vehicle technology, encompassing system categories, essential methodological references, and the fundamental concepts required for comprehending the thesis. It further delves into existing research efforts that address challenges similar to those addressed in this work.
- **Chapter 3** A full description of the implementation of the autonomous driving system and the models that form the core of the thesis is given. Theoretical foundations are established, with focused commentary on the distinct components of individual system implementations.
- **Chapter 4** Presents in detail the methodology of the experiments, the results and the final conclusions. The experiments were executed using the CARLA simulator.
- **Chapter 5** Conclusions derived from the thesis development are explicated in this section, alongside a discourse on encountered obstacles during the creation of the autonomous driving system. Future trajectories, prospective modifications, and potential expansions for augmenting the implemented system are also proposed.

Chapter 2

Literature Review

Chapter 2 aims to introduce concepts, technologies and systems that are fundamental to the understanding of the issues analysed in this thesis. It presents the state-of-the-art technologies in the field of autonomous driving, provides an analysis of the useful concepts covered in the thesis and, finally, reviews the tools and methods to be used.

2.1 State-of-the-art autonomous driving systems

Automated Driving Systems (ADS) are robotic systems that operate in complex and dynamic environments. Therefore, there are many scenarios containing unsolved problems. This section addresses the high-level challenges that exist in autonomous driving in general and aims to present research efforts on key issues related to autonomous vehicles.

2.1.1 Developments in the industry

Many automotive companies have implemented several successful structures in the operation of autonomous vehicles and now their vision is focused on how to build trust between people and these vehicles. Such is the case, for example, with the F 015 [12] research vehicle from Mercedes-Benz, which contains large LED screens at the front and rear, a laser imaging system, and a set of sounds and voice commands, which is used as a Deep Learning-based communication medium. From the same company, we have the Vision Van [13], an electric vehicle used to transport parcels (Figure 2.1).

Tesla company has also made several successful efforts. In recent years it has created the Model S and Model X [14], with several new features, such as automatic lane changing, automatic parking, and an auto-steer system that keeps the vehicle at a constant speed in the lane it is moving in, making use of high-resolution GPS maps, with sensors recognizing the appropriate cues to let the car know when it is in in a lane.

Since 2014 Google has introduced a large number of functional drivers for their Google self-driving car, structuring it in a real-world road environment. The system

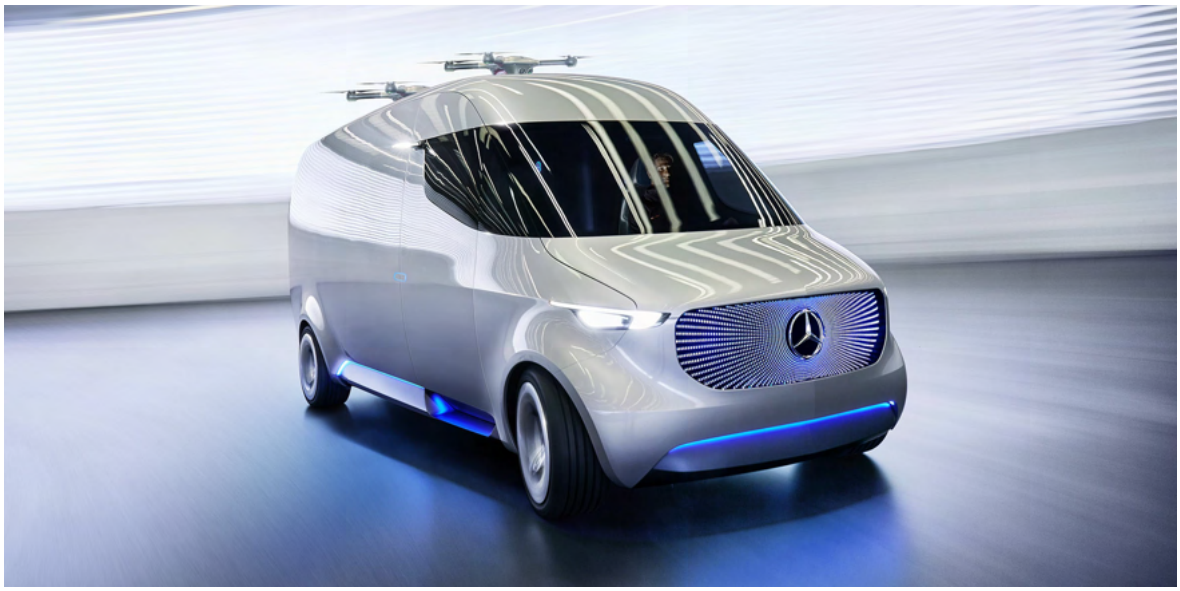


Figure 2.1: Vision Van by Mercedes-Benz [13]

drives the vehicle at the speed limit stored in the maps and maintains its distance from other vehicles using the sensor system. It provides a function that allows a driver to take control of the car, who only needs to apply the brake or spin the wheel, similar to cruise control systems already found in many cars today.

2.1.2 Autonomous Driving Simulators

In recent years there has been a great growth in the field of autonomous driving simulators, as the need to test and evaluate new methods and technologies is increasing. In 2015, Autoware [15], an open source platform used for simulating autonomous vehicles, was created. Its capabilities are specialized for cities, however it also supports highways and other areas. The simulator is built to provide 3D dynamic and static obstacles and supports a multitude of sensors such as RGB cameras, LiDAR, Radar and IMU. It is compatible with ROS software, written in Python and supports only the Linux operating system.

In 2017, Microsoft released the AirSim autonomous driving simulator [16], built on the Unreal Engine game engine. The software in question enabled the testing of Deep Learning and Computer Vision methods in a simulated environment and was an excellent example of how a virtual environment can reduce risk, both for the car and those around it.

In the same year, the CARLA simulator [17] was created by the Computer Vision Centre with contributions from the Toyota Research Institute. This is an open source driving simulator, which relies on the Unreal Engine to run the simulation and uses the OpenDRIVE standard (an open file format for the logical description of road networks) to design roads and driving columns. Control of the simulation is provided via an API (Application Programming Interface).

In 2019, LG [18] is leveraging Unity's leading 3D development platform, namely Unity Simulation. This software allows multiple runs to be performed at scale and is supported by the Google Cloud infrastructure, thereby reducing training hurdles



Figure 2.2: LGSVL simulator environment [18]

and the number of tests. Consequently, it achieves validation of new autonomous vehicle products and services using a virtual environment for their testing. The innovative LGSVL simulator, whose environment is shown in Figure 2.2, can run several driving scenarios in parallel, creating digital replicas of the real world, with precision sensor models for autonomous vehicle training.

2.2 Sensor Systems

The sensing capability in an autonomous vehicle is of primary importance for its reliable and safe operation, as the data obtained from these processes feed into the decision-making process. The sensory instruments used enable the vehicle to make measurements and observations of specific environmental variables. Some sensors enable the distance or position of objects around an area of the vehicle to be measured, while others are used to measure their speed. The sensors used have various advantages or disadvantages depending on the environment and driving conditions for which they are intended. Weather conditions such as rain, snow, sun and fog can greatly affect visibility, which in turn affects performance.

The following are the sensor systems most commonly found in autonomous vehicles. These systems play a crucial role in the vehicle's perception of its environment and are widely used [19].

2.2.1 Camera system

Autonomous vehicles often have cameras and sensors to see and interpret objects around the vehicle, in the same way that humans use their vision. Cameras have the ability to detect objects, classify them and determine the distances between them and the vehicle [20]. For example, cameras can identify cars, pedestrians, cyclists, traffic signs and signals, traffic lights and guard rails.

Cameras are mounted on the roof or on the top of the windscreen to simulate the driver's field of vision, and supplementary cameras can be mounted at the rear of the vehicle. Figure 2.3 shows the common positions of sensors on a vehicle.

Camera as a primary sensing device

One of the most desirable features for autonomous cars is, as already mentioned, the possibility of autonomous navigation. There are many important and well-known examples in this field, such as the Darpa Challenge (2004 and 2005 Grand Challenges and 2007 Urban Challenge) [21] [22] which are among the most famous projects in this field of research.

The basic functions of a navigation system are usually summarized as: localization, mapping, perception, assessment, decision planning and decision making, and navigation through vehicle control [20]. Navigation is related to the ability of the system to correctly collect information about its environment from sensors and process it in order to move through space in a safe and efficient manner. Many of the approaches studied use all sensors and GPS for navigation; however, it is of particular interest to explore the field of space navigation using the camera as the main vehicle sensor.

The vehicle in this case should be able to move within a defined path within the boundaries of the road (navigable area), following the lines, recognizing obstacles it may encounter and following traffic lights and road signs. Using therefore only his visual perception of the space, he should be able to navigate and make decisions about his next movements based on the visual information he receives from the camera's sensing device. These approaches typically use neural networks in combination with an implementation of a state machine (FSM) [20].

The reasons for choosing cameras as a primary sensor in many applications in recent years are many, as they have proven to be one of the most suitable sensor systems for navigating and avoiding obstacles with precision, while their light weight and low energy consumption make them ideal for use in autonomous vehicles [23] [20]. Still, the images can provide a lot of different information about the environment without necessarily needing a combination of sensors, making the process much easier. Finally, costs are significantly reduced by using cameras.

2.2.2 Radar Sensors

Radar (Radio Detection And Range) sensors make a key contribution to the operation of autonomous driving and can greatly facilitate the process. Radar, when operating, sends radio waves to detect objects and then calculates their distance and speed in relation to the autonomous vehicle in real time. They are usually mounted on the periphery of the vehicle and have different functions depending on their range. Unlike camera sensors, which are highly affected by weather conditions and brightness, radar systems usually have no problems in identifying objects.

2.2.3 LiDAR Sensors

The lidar sensor works similarly to the radar in the sense that both sensors emit and receive waves and calculate the time of emission and reception to determine the distance. Lidar (Light Detection And Ranging) sensors send out several targeted laser pulses every second and produce a depth map, or point cloud map, which depicts objects in space in an area around the vehicle. These maps are quite accurate and can be used to give a good prediction of the type of object. They are used to identify objects and to monitor their speed of movement. However, Lidar sensors rely on information derived from the background and therefore cannot detect critical elements without background, such as lights and signs.

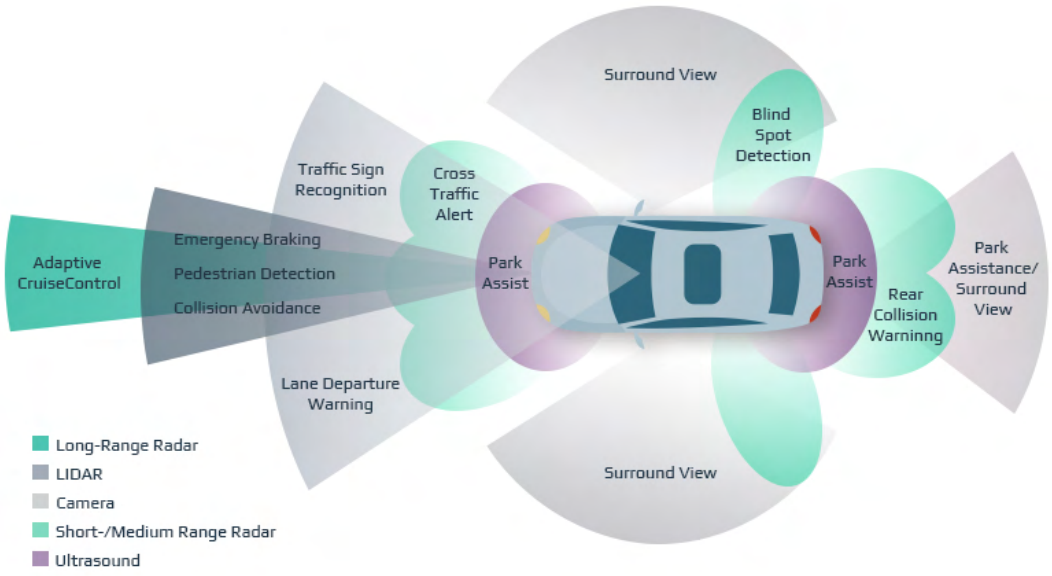


Figure 2.3: Sensor placement on autonomous vehicles [24]

2.3 Digital Image Processing

2.3.1 Image as a two-dimensional signal

The digital image is one of the two-dimensional digital signals that are of considerable interest. The need to process the information in the image leads to a mathematical description of its content. The black and white image can be seen as a two-dimensional function of the x and y space:

$$f \rightarrow f(x, y) \tag{2.1}$$

where f describes a surface, which has a large value when the image is whiter, while for small values of f the image approaches black. In essence, f describes the intensity of the image at each position (x,y). Any image to be digitally processed must first be converted to digital. This process is approximately the same as in the case of a one-dimensional signal. Thus, equal samples of the function f(x,y) must be obtained in the x and y directions. The density at which the samples are taken

is also determined in this case (two-dimensional function in space) by the sampling theorem which is defined as follows:

The sampling rate must be at least twice the highest frequency present in the signal.

We conclude that we should sample fast enough to anticipate the rapid changes in image intensity or brightness. Our samples should then be quantized to a finite number of stations. Typically the weights used are 255 (2^8). A level of 255 corresponds to white, while a level of 0 corresponds to black. Thus, a binary number of 8 bits (1 byte) is sufficient to describe the value of the intensity or brighness of a sample of the image, also called a picture element (pixel).

The digital image is represented mathematically as the width-quantized function:

$$f \rightarrow f_q(n_1, n_2) \tag{2.2}$$

where the discrete spatial variables n_1 and n_2 correspond to the continuous spatial variables x and y. In the general case, the maximum value of n_1 is M and n_2 is N.

Thus, we can think of the digital image as a matrix of M $\times$ N numbers, each of which represents the value of the corresponding pixel.

If the image is in color then at each (n_1,n_2) position, f has three values (vector), which correspond to the values of the colors red, green and blue.

2.3.2 Image Geometry

The image can be seen as a distribution of information (degree of intensity or color) in the plane (x,y). Most optical systems (cameras, video cameras) render analogue images, i.e. images in which the coordinates (x,y) and the degree of intensity f are continuous quantities. In contrast, digital images, which are to be stored in or processed by the computer, have a value only in a discrete number of coordinates (n_1, n_2), which are also the pixel positions. Also, each pixel receives a finite number of intensity values.

The orientation of the coordinates in a digital image may be arbitrary. However, in most cases, and especially in the various commercially available software applications, the coordinate origin (n_1, n_2)=(1,1) is taken at the top left corner of the image. Each pixel has its own coordinates. Therefore, an image having M pixels at n_1 and N at n_2 contains a total of N $\times$ M pixels. The information size of the image is generally large. Even small images contain a large number of pixels, which makes digital processing time consuming. For this reason, image processing algorithms are initially tested on a small part of the image.

2.3.3 Processing

The processing of the digital image generally involves many different steps depending on the format and initial state of the digital image and the desired end result.

The actual image will be recorded by the digital image acquisition system. This will be followed by a noise filtering and restoration stage to correct errors caused either by the medium between the image and the acquisition system or by the acquisition system itself. Errors that occur in the image during its acquisition are noise and/or spike noise (from the acquisition system) and image distortion.

Image enhancement is mainly focused on highlighting specific elements of the image in order to facilitate the tasks that will follow. The final stage is computer vision. This includes all automated systems for image processing, object recognition and decision-making for specific actions.

2.4 Machine Learning

2.4.1 Introduction

Machine Learning is a subfield of Artificial Intelligence, aiming at the ability of machines to mimic intelligent human behaviour in order to solve complex problems. It was originally defined by Arthur Samuel in 1959, describing it as 'the field of research that enables computers to learn without being explicitly programmed to do so' [25].

Machine learning algorithms use data (numerical, text, image, image, sound, etc.) and "learn" from it to make predictions. The training data must be sufficient in both quantity and quality for successful training of a model. The programmers' job is to provide the model with appropriate data (including pre-processing techniques), to select the appropriate model and to parameterise it, depending on the particularities of the problem to be solved. The 'machine' then takes over, responsible for training the model and finding patterns or making predictions. In addition to the training data, a piece of data must be captured to be used as a test set. The results of the test process can provide us with very useful metrics for how well the model behaves. It is quite common to divide Machine Learning models into the following three categories [26]:

- Supervised learning
- Unupervised Learning
- Reinforcement learning

Applications of Machine Learning involve computational tasks, where it is possible to design and implement algorithms in a programming environment. Examples of applications include spam filtering, optical character recognition (OCR), search engines and computer vision. Machine learning is even confused with data mining for the purpose of exploring and analysing data, also known as unsupervised learning. Machine learning is also applied to the field of data analysis and is a method used to create complex models and algorithms used to predict outcomes. Finally, two fundamental approaches to machine learning are artificial neural networks and deep learning.

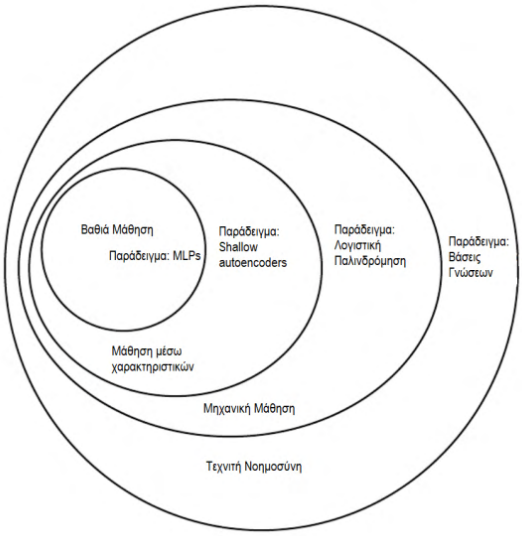


Figure 2.4: AI Disciplines in a Venn Diagram. [27]

2.4.2 Deep Learning

Deep learning is part of a larger family of learning methods based on artificial neural networks. The term "deep" refers to the number of layers in the network through which the input data passes until the result is extracted. Learning may be supervised or unsupervised. These neural networks attempt to simulate the behaviour of the human brain by allowing it to "learn" through the use of supervised learning a large volume of data. While a neural network with a single layer can make approximate predictions, additional hidden layers within it can help optimisation and accuracy. Deep learning architectures such as deep neural networks, iterative neural networks and convolutional neural networks have applications in various fields such as computer vision, speech recognition, natural language processing, audio recognition, social network filtering, medical image analysis, autonomous driving, etc.

More specifically, in deep learning, each level learns to transform its input data into a slightly more abstract and complex representation. The lower layers can identify the edges of a shape, while the higher layers can recognise concepts that humans understand, such as digits, letters or faces. The necessary features are modelled and understood by the neural network on the basis of the data it has been trained. The most important difference in the learning process, compared to simple neural networks, is that the network alone can learn to assign the extraction of features to each layer in an optimal way.

Most modern machine learning models are based on artificial neural networks, and a very important class is convolutional neural networks or more commonly known as Convolutional Neural Networks (CNN)[28].

2.4.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a class of neural networks commonly used in the analysis of visual images. The name convolutional neural network im-

plies that it uses a specialised type of linear operation called convolution, which is used in generalised multiplication at at least one level. Convolutional layers have Kernels or Filters, which have proven to be very efficient in detecting features and patterns in optical data. Convolutional neural networks are a grid-like neural network topology inspired by biological processes, the pattern of connectivity between neurons and resembling the organisation of the visual cortex. Each individual neuron responds to a limited area of the visual field called the receptive field. All the receptive fields of the neurons contribute to the coverage of the entire visual field.

Convolutional neural networks are a normalized form of MLPs (Multi Layer Perceptrons) and are usually fully interconnected. This characteristic makes them prone to the phenomenon of overfitting. Typical ways of dealing with this phenomenon include measuring loss functions. However, convolutional neural networks take a different approach, which exploits the hierarchical pattern in the data by breaking down more complex designs into smaller and simpler patterns. As a result, CNNs find themselves low on the scale of connectivity and complexity.

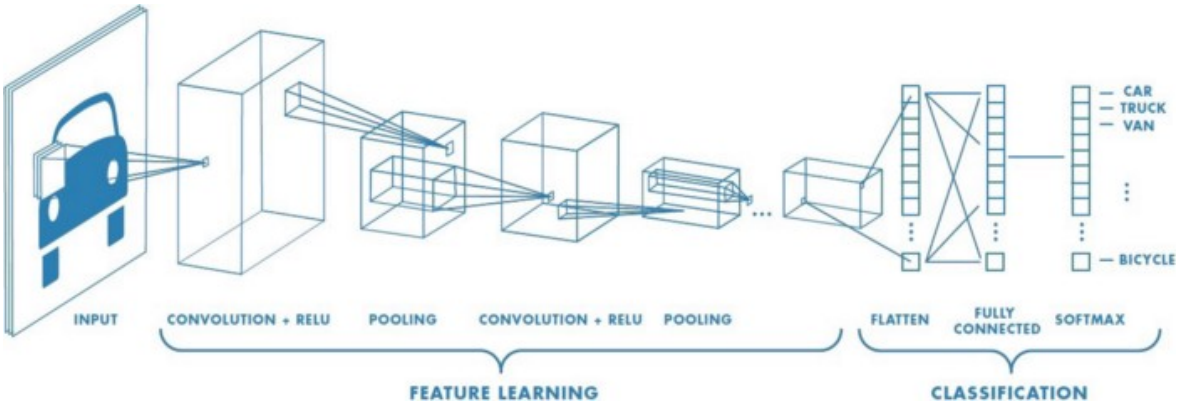


Figure 2.5: Convolutional Neural Network Architecture [29]

CNNs use relatively little preprocessing compared to other image classification algorithms. This is because the network learns the files on its own, unlike other algorithms where the determination of the files is done by humans based on their knowledge. This independence from prior knowledge and human effort in designing neural network features is a significant advantage. Figure 2.5 shows a representation of the structure of a CNN.

Convolutional Neural Network Architecture

A convolutional neural network consists of an input layer, an output layer and several hidden layers. The hidden layers are the ones that contain the convolution process. The activation function of these layers is usually ReLU and is followed by additional convolutional layers such as pooling layers, fully connected layers and normalization layers. These layers are called hidden as their inputs and outputs are covered by the activation function and the final convolution. The final convolution often includes the technique of backpropagation where a specified loss function corrects the errors to improve the result.

Convolution

When constructing a CNN, each convolutional layer contains the following characteristics:

- The input is a vector of the form (number of images)×(image width)×(image height)×(image depth).
- Convolutional neurons have hyperparameters, the width, height and depth of which must be equal to that of the image.

The convolutional levels process the input and carry its result to the next level. Although fully connected feedforward neural networks can be used to learn features as well as classify data, it is not practical to apply this architecture to images. A very large number of neurons would be necessary due to the large input sizes associated with images, where each pixel is a relative variable. For example, a fully connected layer for a small 100×100 image has 10000 weights for each neuron in the second layer. The convolution process (Figure 2.6) provides a solution to this problem as it reduces the number of free parameters, allowing more layers with fewer parameters [30].

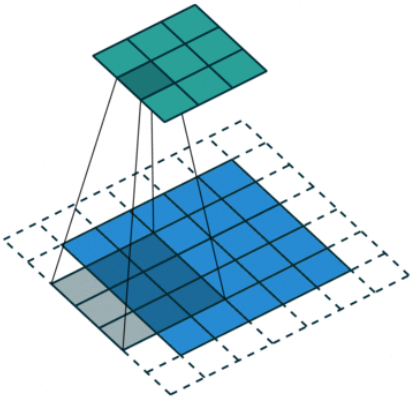


Figure 2.6: Convolution Process [29]

The convolutional layer is the central building block of a convolutional neural network. The parameters of the layer consist of a set of mathematical filters. During forward propagation, each filter traverses the width and height of the input values, creating a two-dimensional activation matrix for that filter. As a result, the network learns filters, which are activated when it detects a particular type of feature at a certain spatial position in the input. The placement of the activation maps for all fillers along the layer constitutes the complete output of the convolution layer.

Pooling Layer

Convolutional networks may include pooling layers, which are applied either globally, affecting all neurons in the network, or locally at a single layer [31]. Pooling layers, in a topical application, reduce the dimensions of the data by combining the outputs of the neuron arrays in one layer into a single neuron in the next layer. In addition, the combination can be applied to calculate a maximum or average. In the

case of the maximum, the combination uses the maximum value from the previous layer to the next layer, while the average uses the average value for the next layer. In these layers the features extracted from the previous convolution layers are gathered and combined, thus constructing a feature map that will be used by the subsequent layers for further analysis of the image.

Intuitively, the exact location of a feature is less important than its position relative to other features. This is the idea behind the use of combination in convolutional neural networks. The combination layer serves to gradually reduce the spatial size of the representation of the input information, reduce the number of parameters, the memory footprint and the number of computations in the network, and hence control overload. It is common to periodically introduce a layer of combination between successive convolutional layers in a CNN architecture. As we will see below, the way these maps are extracted is very important since it determines the speed and accuracy of the network.

Fully Connected Layer

Fully connected layers connect every neuron in one layer to every neuron in another layer. Its finance is the same as the traditional multilevel perceptron neural network (MLP). In convolutional neural networks it is used to classify the output according to the set of features extracted from the previous layers and therefore found in the last layers of the network. Essentially it is what s the network's final decision on the classification of the image content. However, a CNN may or may not contain a fully connected layer at the end.

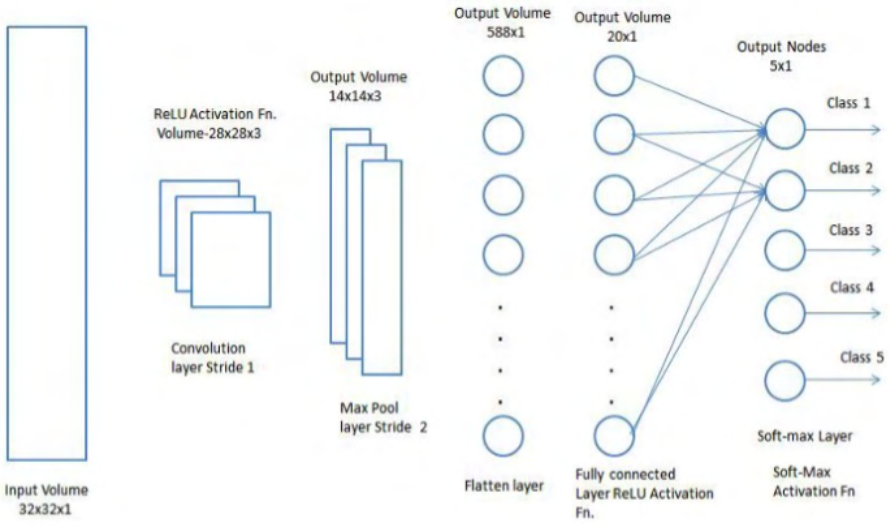


Figure 2.7: Fully Connected layer [29]

Normalisation

The normalisation is used to improve the speed, performance and stability of an artificial neural network. It can be referred to as batch normalization as it appropri-

ately modifies the values in its input to pass to the next level in the optimal way. The need for its use arose from the problem of the difference between the values of the parameters in the levels during training. Thus the network has to adjust the values at each distribution so that there are no large deviations between levels.

Particularly in deep learning networks, small parameter changes in the first few levels can create large changes in the latter levels. By the introduction of inter-level normalisation, these changes are cancelled out, leading to the creation of more reliable models. The network can also be trained at a higher learning rate without losing feature detail. Finally, it is easier to build a model that learns general features without using the dropout method, which discards a portion of the values during training, leading to the possible loss of fine features.

Loss - Activation Functions

The loss function determines how the training imposes the penalty on values that deviate from the desired output and is usually executed at the last level of the neural network. The choice of the appropriate loss function depends on the purpose and the process being performed. The most familiar function is SoftMax, which is used to predict a single class of mutually exclusive classes. The sigmoid function is also used to derive satisfaction estimates for all independent classes.

The activation function of a node determines the output of that node with a given input or set of inputs. The most common activation function is ReLU, which effectively removes negative values from an activation map by setting them to zero. It increases the nonlinear properties of the decision function and the overall network without affecting the convolution layer's receptive fields. ReLU is often preferred over other functions, as it trains the neural network several times faster without a significant penalty in generalization accuracy.

2.5 Computer Vision

2.5.1 Introduction

Computer vision, computer vision or artificial vision, which is a field of Artificial Intelligence, is a method of automating data analysis. It is based on the ability of systems to learn from data, recognise patterns and make decisions with minimal human intervention. It attempts to reproduce algorithmically the sense of vision, usually on a computer or a computer bot. Computer vision is related to the theory and technology involved in the design and construction of systems that analyse data from digital images. This data can be photographs, videos, views from multiple cameras, multidimensional images, etc.

Machine learning has emerged as a useful tool for modelling problems that are otherwise difficult to formulate accurately. With machine learning, part of the human input is replaced by a learning algorithm. The availability of computing power and data has increased, with the result that machine learning has become increasingly practical and necessary over the years.

2.5.2 Preprocessing

The organisation of a computer vision system depends to a large extent on the application to be implemented. Some systems are stand-alone applications aimed at solving a specific problem, measurement or detection, while others are a subsystem of a larger design which includes, for example, subsystems for the control of mechanical actuators, design, databases, mechanical interfaces, etc. There are typical functions applicable to many computer vision systems. Image acquisition is the first function in the sequence and concerns the digital image produced by one or more image sensors. The pixel values usually correspond to the intensity of light in one or more optical bands (gray or light-colored images), but may also be related to various physical measures such as depth, absorption or reflection of sound or electromagnetic waves.

Preprocessing is an operation that involves the processing of data to ensure that it satisfies certain assumptions. These conditions are indicated by the method that computer vision will use on the image data in order to extract some specific information. Some **preprocessing techniques** are as follows:

- Repeat the sampling to confirm the correctness of the image coordinate system.
- Noise reduction to ensure that sensor noise does not introduce false information.
- Image contrast enhancement to ensure detection of relevant Image resizing to appropriate dimensions.

The **extraction** of features from image data. Examples of such features are:

- Lines, edges and ridges.
- Regions of interest, such as corners, spots or points.
- More complex features may be related to texture, shape or movement.

Detection/segmentation refers to the decision made during processing about which points or regions of the image are important for further processing. Examples are:

- Selection of a specific set of regions of interest.
- Segmentation of one or more image regions containing a specific object of interest.
- Segmentation of the image into scenes including foreground, groups of objects, individual objects or dominant objects.

High-level processing is where the input is usually a small set of data, for example a set of points or a region of an image that is thought to contain a particular object. This processing involves, for example:

- Verification that the data satisfy assumptions made in models and applications.
- Estimation of specific application parameters, such as object presentation or object size.

- Image recognition/classification of a detected object into different categories.
- Image entry - compare and combine two different views of the same object.

Decision-making that involves taking the final decision required for implementation, for example:

- Pass/fail in automatic inspection applications.
- Matching or mismatching of identification applications.
- Note for further consideration by human operators in medical, military, security and reconnaissance applications.

2.5.3 Object Detection

Object detection is a major area of study in recent years in the field of computer vision. The main problem in computer vision is image processing and determining whether an image contains objects, features or activity. The most well-known recognition problems described in the literature are:

- Object Classification involves one or more identified objects or classes that can be identified.
- The identification or determination of the category to which an identified object belongs.
- Detection involving the scanning of an image relative to a condition.

In many cases, small areas of potential interest are detected, to which more computationally demanding algorithmic techniques are then applied to determine the final conclusion.

In recent years, due to the rapid growth of research in the field of autonomous driving, many different models for the object recognition process have been presented which provide high accuracy in their predictions and are widely used [32] [33]. The most efficient algorithms for such tasks are based on convolutional neural networks. The most prominent representatives of the capabilities of these networks are the ImageNet Large Scale Visual Recognition Challenge [34]. This is a benchmark in object classification and detection, with hundreds of classes and millions of images. The performance of convolutional networks in the ImageNet trials approaches that of humans.

Previously, traditional methods did not include ANNs. The most influential of their time were Viola-Jones [35], HOG [36] and DPM [37]. The majority of these models, however, used similar techniques: they did not directly use the RGB values of the image pixels as input features of the categorization, but possessed special mechanisms to extract important features (feature extractor) from the image. The Viola-Jones (V-J) method [35] was developed in 2001 by P. Viola and M. Jones and was originally used for human face detection. The element that made it stand out at that time was the extraction of Haar-like features from the image and the use of the AdaBoost algorithm which reduced the feature set to the most representative ones. The Haar-type feature values are defined by the variations in color intensity in

different adjacent regions of the image, such as contrasts between bright and dark regions. Thus, it becomes possible to perceive shapes such as lines and angles.

In 2005, the Histogram of Oriented Gradients (HOG) method was proposed by N. Dalal and B. Triggs [36]. Similar to the V-J method, it does not directly use color intensities as features, but by dividing the image into subregions, it extracts the histograms of oriented gradients from each one. The extracted features of each subregion are then fed into a Support Vector Machine (SVM) classifier [38]. Figure 2.8 shows an example of the use of the method. The use of oriented gradients proved to be more accurate, but was slower than using Haar-type features in the localization problem. This led to the development of localisation systems that combine both families of features [39], gaining the advantage of both accuracy and speed.

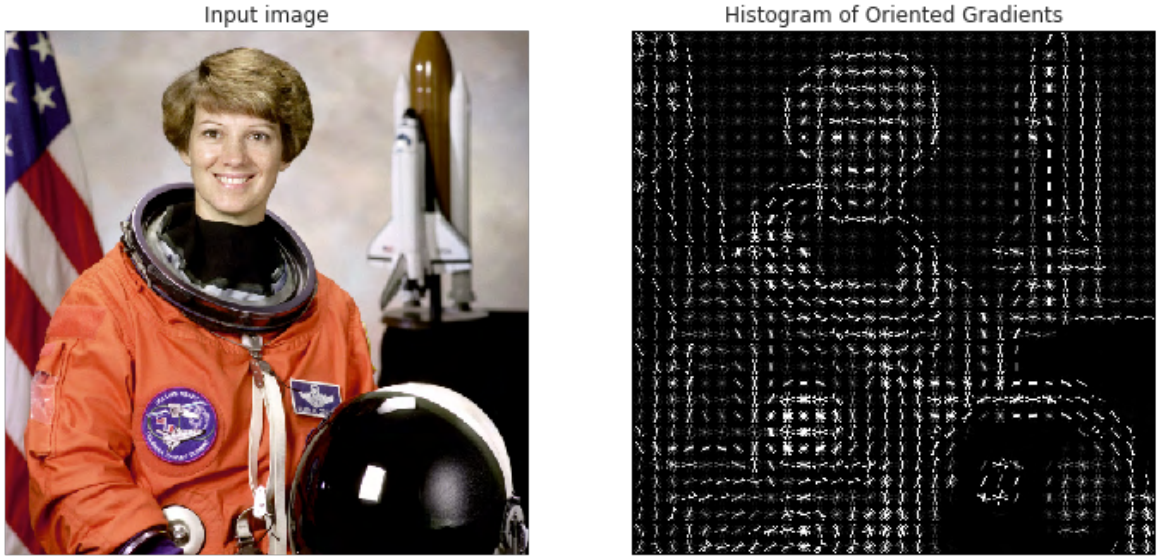


Figure 2.8: Histogram of Oriented Gradients example [40]

The Deformable Part Model (DPM) [37] is an improved version of the Deformable Part Model (DPM). HOG method. It was proposed by P. Felzenszwalb et al. in 2008 and subsequently underwent significant improvements by R. Girshick that increased the detection speed by 10 hours while keeping the accuracy constant. The excellent performance of the algorithm was demonstrated in the Pascal Visual Object Classes (VOC) competition [41] where it was applied to a dataset of common objects and was a reference for multiple winning systems in the period 2008-2012.

In the period that followed, the traditional exporters of the period were unable to solve more complex problems. In 2012, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) competition [34] spurred the development of new characterization methods that had DNNs as a main component. The goal of this competition was to recognize classes of objects from the ImageNet [42] large-scale dataset, one of the most complex and large datasets of its kind. The winners of the competition, A. Krizhevsky, I. Sutskever and G. Hinton, developed a CNN called AlexNet [43], the structure of which is shown in Figure 2.9. Initially, the use of maximum value pooling in overlapping regions of the image was crucial. AlexNet made learning possible by using the computational power and memory of multiple GPUs, a first for the time. The promising performance of CNNs in class recognition paved the way

for the development of new variants with the ability to not only recognize but also localize.

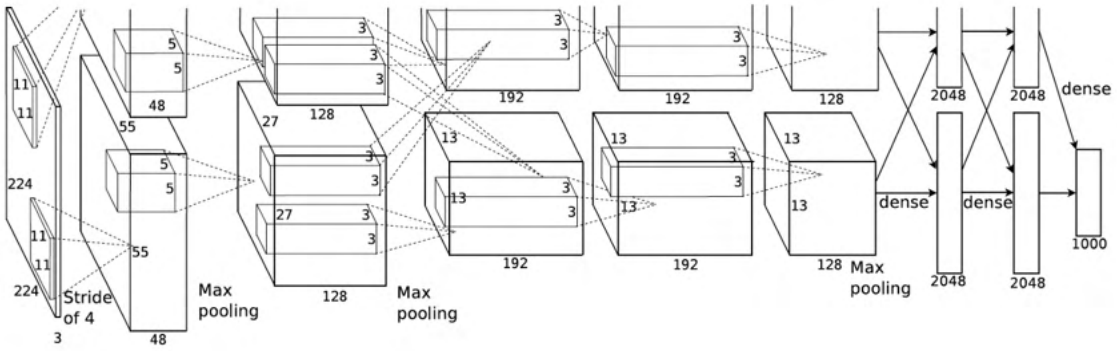


Figure 2.9: AlexNet Architecture [43]

Object detection can be grouped into two subtechniques, object localization and object classification. Object localization is the technique whereby all objects to be categorized in the image are identified.

There are several algorithms for object detection and they can be divided into two categories. The detection of a single instance of the class from the image is called single class object detection, while the detection of the classes of all objects present in the image is known as multi class object detection.

Furthermore, modern object detection and recognition models are divided into two categories depending on how predictions are made at the network head: two-stage object detectors and one-stage object detectors. In two-stage detectors, the model starts with region proposals, i.e. regions with a high probability of containing an object, and then the classification of the object is performed using convolutional neural networks. In contrast, in one-stage detectors no region proposals are generated and the model performs class recognition and localisation directly from the features of the whole image, treating the process as a regression problem. One-stage models, although renowned for their high speed, tend to exhibit lower accuracy rates than two-stage models, but are better suited for real-time object recognition.

Among the most well-known two-stage models is the R-CNN (Region-based Convolutional Neural Network) proposed in 2014 [44][45]. The methodology of the model is simple. First, region of interest suggestions are extracted in the image using the Selective Search algorithm. The algorithm performs an initial segmentation of the image according to P. Felzenszwalb's algorithm, based on the similarity in color, texture, size and shape between subregions of the image, and iteratively merges neighboring segments with similar properties. The final segments are delimited into frames that will form the proposed regions of interest. These proposals are subregions of the image with a high probability of containing a candidate object. Each candidate region is then inserted into a CNN with the aim of extracting significant features in vector form. The feature vectors are fed into an SVM classifier which is responsible for assigning a class to the delimited object. Finally, a regression model is used to refine the accuracy of the surrounding boxes around the object, minimising the error. The structure of the R-CNN is shown in Figure 2.10.

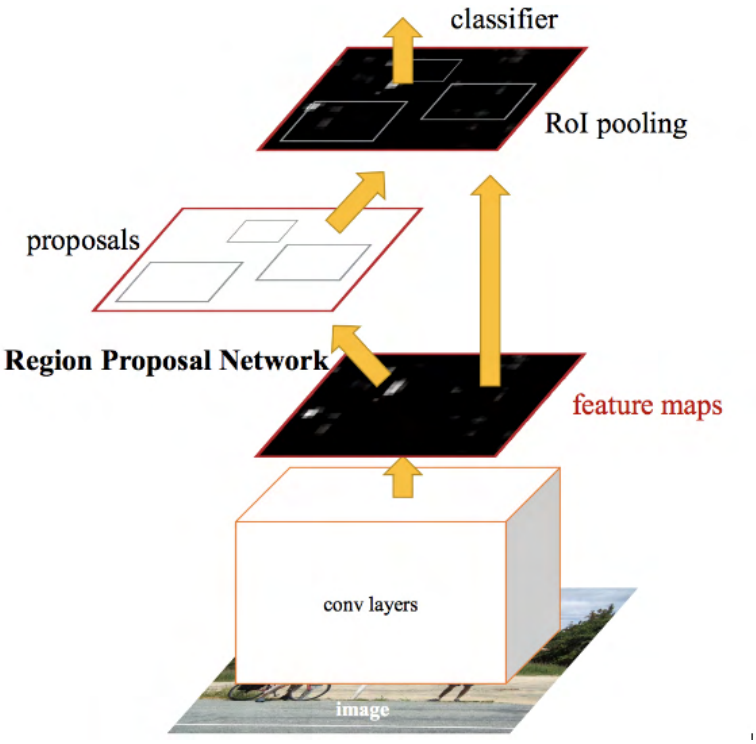


Figure 2.10: RCNN Architecture [44]

However, the R-CNN had some important disadvantages. Firstly, due to the large number of proposal regions generated by the network, the introduction of each of them into the CNN increases the cost both in terms of localisation time and storage space for the feature vectors of each region. In addition, the CNN is limited to inputs of a certain size due to the SVM (or fc-level) classifiers that follow the conv levels. Thus, each region of interest is subject to normalization to the appropriate dimensions, which affects the image quality significantly. For these reasons, Fast-R-CNN [46], FasterRCNN [47] and Mask-RCNN [48], which are variants of the simple R-CNN, were proposed.

In 2015 J. Redmon et al. developed the first one-stage detector model for real-time object detection, You Only Look Once (YOLO)[49]. Its creators followed a subversive tactic in terms of the detection process. In particular, the localization of the surrounding boxes and the object recognition were achieved simultaneously in a CNN which produced the coordinates of all boxes and their class probabilities in the output with a readout of the whole image, hence its name. The architecture of YOLO is illustrated in Figure 2.11. The logic followed by YOLO was simple. First, the image is divided into a square grid of k $\times$ k cells. Through the sons layers, each cell is reduced to a vector of M features, the set of which is inserted as a k $\times$ k $\times$ k $\times$ M tensor into the two successive fc layers of the grid for the predictions. The network predictions were of two types, class detection and class recognition. For localization, each cell was responsible for delineating the object whose center was within the cell. Thus, for each cell a prediction of a set of surrounding boxes of the object was generated.

YOLO was extremely faster (6$\times$ faster than Faster R-CNN) and at the same time similarly accurate (mAP 63.4% in VOC2007) to other state-of-the-art models of the

time. Instead of techniques for suggesting regions of interest, YOLO used information from the entire image to predict the location and class of all objects. Thus, it formed a more global picture of the appearance of objects, generalising better to new data compared to Fast RCNN, which incorrectly identified some 'non-objects' from the image background more often than YOLO. However, it had difficulty in detecting very small and densely placed objects as only one object per cell could be detected. In subsequent versions of YOLO, v2, v3, v4 and v5, J. Redmon et al. attempted to solve the problem.

W. Liu et al. proposed in 2015 the Single Shot MultiBox Detector (SSD) model [50], the second one-stage detection model after YOLO. SDD differed from the other detection models of the time in that it produced for each image maps of characteristics at numerous size scales and then used all of these representations for detection, a technique that was improved along the way to FPN networks while using in-process oversampling. This allowed for high accuracy in the detection of images of various resolutions, even very small ones. Moreover, the predictions of the surrounding boxes of each scale and their classes were not made from fc planes but from conv planes and were extracted in the form of displacements from some predefined initial boxes. For each object, multiple candidate displacements were predicted, from which the one with the highest confidence score was selected as the final box environment using a Non-Maximum Suppression (NMS) algorithm. SDD was faster and more accurate than the first version of YOLO and as accurate (mAP 74.3% in VOC2007), if not more so, than other two-stage detector models, such as Faster R-CNN. Figure 2.11 shows the differences in the structure of YOLO and SSD.

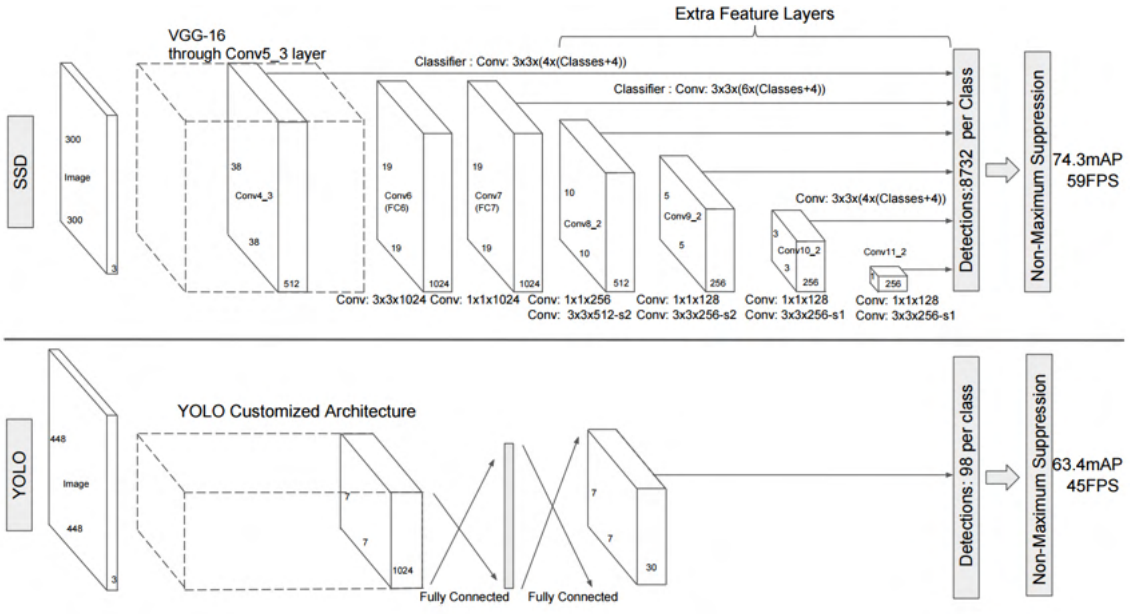


Figure 2.11: Contrasting YOLO Architecture with SSD [50]

Figure 2.12 summarises the above methods in terms of their performance in terms of accuracy (mAP) and speed.

Some challenges that need to be handled when performing detection of an object are partial/full occlusion of the object, different detection lengths, camera positions, scaling, etc. In general, object detection is the first step in any visual recognition

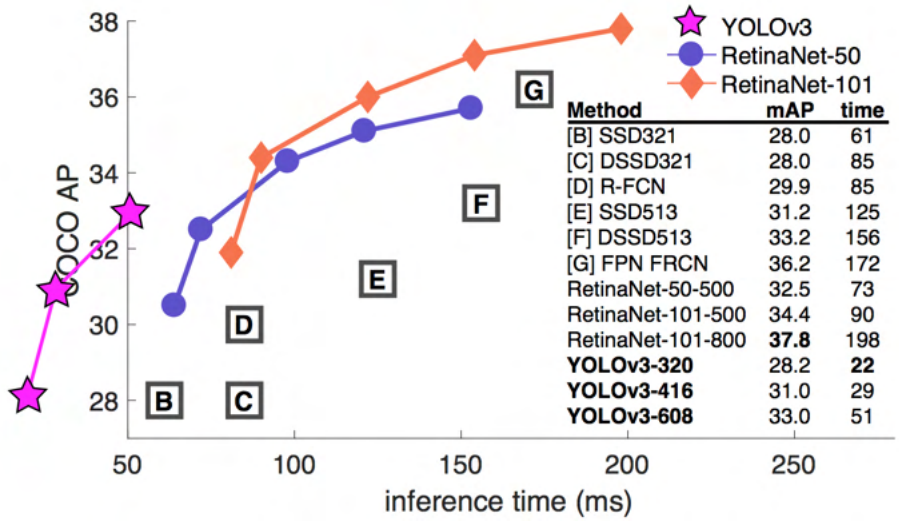


Figure 2.12: Performance Evaluation of Object Detection Methods [51]

activity.

2.6 Algorithms and tools

2.6.1 CARLA Simulator

CARLA [17] is an open source driving simulator. It was built to be used as an application programming interface (API) for autonomous vehicles and to perform a number of tasks involved in the autonomous driving problem, in which the user can change and control parameters in the simulation. It is built on the Unreal Engine 4 game engine to run the simulation and uses the OpenDRIVE (open file format for the description of road networks) standard for the design of roads and driving rules. The versions have been constantly updated. Version 0.9.13 (CARLA 0.9.13) is the one used in this thesis (the newest version) which was released in 2021 and includes all the necessary features needed for this thesis [52].



Figure 2.13: CARLA simulator [52]

One of the main objectives of the CARLA simulator is to enable more users to become involved in autonomous driving, to be used as a tool that is easy to access and adapt by users. To do this, the simulator must meet the requirements of different use cases in the general driving problem (e.g. learning driving policies, perception training algorithms, etc.). Among other things, the user can specify the road environment (Town01 - Town10), change the weather conditions, the number of vehicles and pedestrians present in each scenario, etc. through the interface. Control of the simulation is provided through an interface (API), which can be used with Python and C++ programming languages. The PythonAPI available includes a GUI created with PyGame with which the user can control the manually selected vehicle. For the present thesis, the possibility of changing weather conditions, light and field of view in front of the vehicle is quite important, which particularly affects the perception of objects in the environment and can make predictions easier or more difficult. In this study, an RGB camera was placed at the front, in the middle of the vehicle, so that the field of view it records is similar to that of the driver.

Weather and **lighting conditions** can be provided by the user or selected from a set of predefined settings [52]:

0 - Default

1 - ClearNoon

2 - CloudyNoon

3 - WetNoon

4 - WetCloudyNoon

5 - MidRainyNoon

6 - HardRainNoon

7 - SoftRainNoon

8 - ClearSunset

9 - CloudySunset

10 - WetSunset

11 - WetCloudySunset

12 - MidRainSunset

13 - HardRainSunset

14 - SoftRainSunset

It also includes **10 Town maps** with different layouts to suit each scenario that the user may wish to run. The term **Actors** in the CARLA simulator refers to any element performing actions within its environment, i.e. vehicles, pedestrians, sensors, traffic lights and traffic signals. The simulator has a wide variety of actors, as the user can choose between 26 vehicles, a large list of sensors, bicycles, motorcycles and pedestrians of all ages. The **vehicle physics** is quite realistic, such as in the collision of vehicles and when the vehicle is sliding on a wet road. It has the possibility to control the physical characteristics of the vehicle, such as vehicle weight,

maximum speed, friction with the road surface, etc. The Python API can also be used to design specific **driving scenarios**. CARLA allows the exact same scenario to be executed several times, giving exactly the same data from the sensors and the states of the vehicles [52].

In CARLA, there are also **waypoints** on the map at each possible vehicle location. For each lane, these waypoints have a certain distance between them. For each waypoint, CARLA provides a lot of information, such as position and orientation of the waypoint according to its lane, road identification and lane information via the Python API. The lane information, consists of lane identifier, lane width, intersection information (whether left and/or right turning is allowed), type (parking, sidewalk, crosswalk, etc.), right and left lane marking type (solid, dashed). In addition, the Python API includes methods that return the next or previous waypoints. If two points are closer than a threshold, they are considered as one by CARLA.

While in autopilot mode, for each vehicle and pedestrian CARLA shall generate a list of random waypoints and select one of them as the first destination. Each time an actor reaches its destination, CARLA randomly selects another one from the list or creates new locations if the list is empty. When moving, if there is only one point in front of the actor, then the actor moves to that point. If there are more points in front which means that the actor has to choose between different lanes or roads then some path planning algorithms are used to select the appropriate one.

2.6.2 OpenCV Library

The image processing in this thesis was achieved using the OpenCV (Open Source Computer Vision) library. The library is platform independent and implements computer vision functions compatible with Python, C, C++, Java and Matlab as well as on Windows, Linux, Android and Mac operating systems. It contains more than 2500 algorithms and is designed with high computational efficiency in mind and is used in many applications, mainly real-time, such as facial recognition systems, advanced trading systems, motion detection systems, etc [53] [54].



Figure 2.14: OpenCV [53]

OpenCV contains more than 500 functions suitable for many different applications in the fields of optical processing. It also has the Machine Learning Library (MLL), which consists of various functions and tools that contribute to statistical classification, regression and clustering of data, as well as functions for viewing and storing videos and everything else related to the user-application interface. Its widespread use combined with its simplicity of use and the fact that it is open-source

make it very attractive and are the reasons for its selection in this thesis. The version used is 3.3.0, which was implemented in 2017.

2.6.3 Hough Transform

The Hough Transform [55] is a technique used to isolate certain features of a particular shape in an image, lines, circles and other simple shapes. Duda & Hart introduced it to image processing and computer vision. In the present interpretation, the emphasis is on the Hough Lines Transform since the detection is concerned with the road lanes and pavements, which are straight lines. In particular, we study the detection of straight lines using a "high-order" procedure. The central idea is to transform the problem of detecting lines into a simpler problem of detecting maxima in the parameter space of the line, called Hough Space. First, an edge detection technique such as the Canny Edge Detector is applied, as described in detail in the next section [56]. The fixed theoretical thinking behind the Hough Line Transform is that each point-pixel in a binary image may be part of some line. A line in image space can be expressed by two variables in either the Cartesian Coordinate System or the Polar Coordinate System. To use the transformation, we need a way to express the line mathematically. Specifically,

$$y = mx + b \tag{2.3}$$

where m is the slope of the line and b is the intersection of the line with the y-axis. However, when the lines tend to become perpendicular, the quantities m and b tend to infinity. Therefore, a more useful representation of a line is by means of polar coordinates:

$$xcos\vartheta + ysin\vartheta = \rho \tag{2.4}$$

The equation expresses a line passing through the point (x, y) and is perpendicular to the line starting from the origin and ending at the point (ρ, ϑ) of the polar coordinates, i.e., $(\rho cos\vartheta, \rho sin\vartheta)$ in Cartesian coordinates, as shown in Figure 2.15.

In a line, ϑ and ρ at each point are constant. For any given point (x, y), we obtain the lines passing through it by solving for ϑ ρ. Using possible values of the angle ϑ, we can solve using the equation directly as to ρ [55]. A grid is created and specified from the parameters (ρ, ϑ) and the selected step. A value is then accumulated at each point of the grid, which indicates how well an object obtained from the parameters specified for that point corresponds to the processed image. In the grid, the points that accumulate large values are those that describe features that have the potential to be projected onto the original image, matching to a certain extent features that actually exist in the image.

Assume that to find the Hough transform of the lines of an image, a particular angle is chosen, and a switch is made between the angles specified in that step. For each angle, we ϑ solve for ρ:

$$\rho = xcos\vartheta + ysin\vartheta \tag{2.5}$$

Then, the value at (ρ, ϑ) is increased. This process can also be described as voting the point (x, y) for the line determined by the coordinates (ρ, ϑ). There are several variations on how the votes are generated to make the transformation more stable.

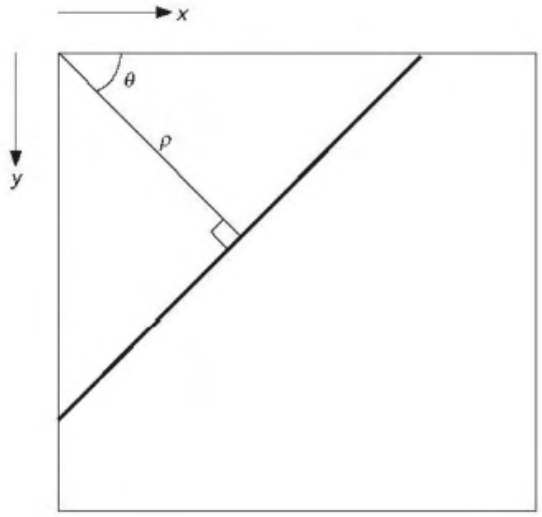


Figure 2.15: Polar Coordinates in Hough Transform [57]

At the same time, in order to detect a line, a threshold value is required, which determines the minimum value needed to consider a line to be formed.

2.6.4 YOLO and Darknet

YOLO (Joseph Redmon, 2016) [49] is an algorithm that uses neural networks to provide real-time object detection and recognition. This algorithm is well known due to its speed and accuracy. It has been used in a variety of applications for the detection of paintings, people, vehicles, animals, etc.

This algorithm uses convolutional neural networks for real-time object recognition. It is characterised by its name You Only Look Once which indicates that during both detection/recognition and training, the network receives the input image once and does not process it again after the input during its execution. This means that the prediction on an entire image is done within one cycle of the algorithm. For YOLO, the detection is a simple regression problem that takes an input image and returns the probabilities of the classes and the coordinates of the bounding box. In this thesis the YOLOv4 model is used.

This algorithm was chosen for the following reasons:

- **Speed:** Detection time is improved because it can make real-time predictions.
- **High accuracy:** provides accurate results with minimal background errors.
- **Learning opportunities:** The algorithm has excellent learning capabilities that allow it to learn the appearance of objects and apply them to object recognition

Because the input is processed once, the neural architecture consists of convolution, sub-sampling and two totally connected layers at the end. As shown in Figure 2.16, YOLO divides each image into an S x S grid, and each grid provides N bounding boxes and a confidence score. This score reflects the accuracy of the frame and whether the frame actually contains an object, regardless of the class into which we want to categorize the objects. YOLO also provides the confidence of the categories for each bounding box, for each category at the stage of learning. Therefore, a total of S x S x N frames are predicted. However, most of these frames have a low confidence rating. By setting a desired threshold value, any frame with a value greater than the threshold is selected and used to find an object.

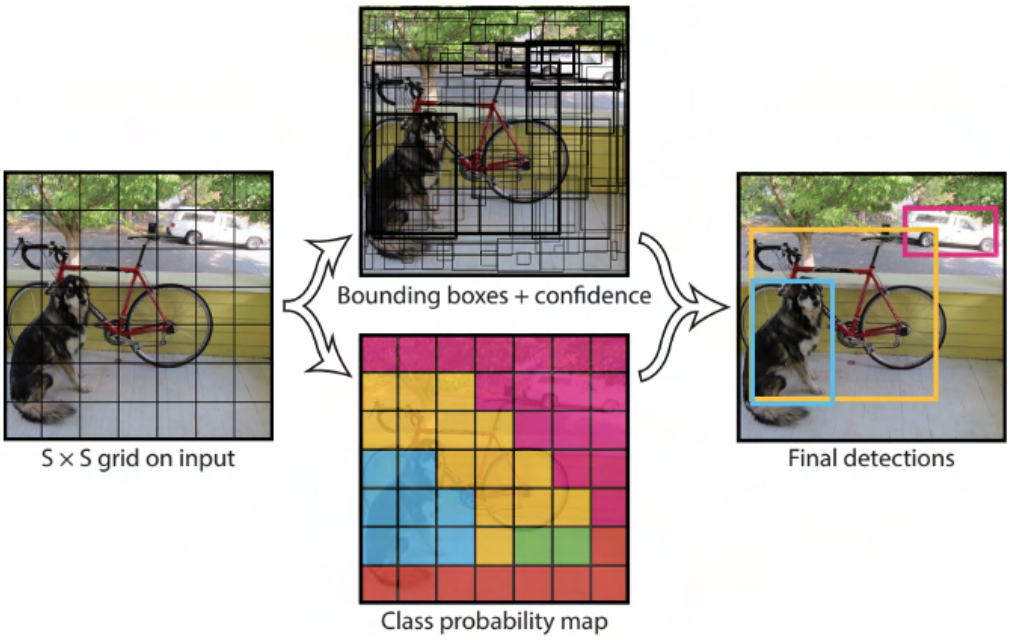


Figure 2.16: YOLO Conceptual design [29]

The information describing a **bounding box** is (x, y, w, h, confidence score), where

(x, y): the centre of the bounding box

w: rectangle width / image width h

h: rectangle height / image heigh

Moreover, each cell has C probabilities $Pr(Class_i \mid Object)$, i = 1, .., C.

As activation function, instead of the most common ones, a similar form is used:

$$\varphi(x) = \begin{cases} x, & x > 0 \\ 0.1x, & \text{else} \end{cases} \tag{2.6}$$

The **IoU (Intersection over Union)** metric quantifies the degree of overlap between two bounding boxes. IoU evaluates the overlap by calculating the ratio of the overlap area to the combined area of prediction and ground truth. The IoU is

equal to 1 if the bounding box found is the same as the actual bounding box. This mechanism rejects bounding boxes that are not the same as the actual ones. Figure 2.17 shows some examples on the evaluation of the metric.



Figure 2.17: Intersection over Union [58]

As mentioned above, the algorithm runs through the image only once. Therefore, YOLO is orders of magnitude faster (45 frames per second) than other object detection algorithms. Another key difference is that YOLO examines the full image simultaneously as opposed to examining only the region proposals observed in other methods. Thus, this information provided by the algorithm helps to avoid false positives. Another case is that while it tries to generalise for small and large objects, when they are found in images with different aspect ratios or repeated in a group with small dimensions it fails to detect them. The system developed in this dissertation will be implemented with the YOLOv4 algorithm.

2.6.5 Tesseract OCR

Optical Character Recognition or Automatic Text Character Recognition is the process of converting clear images of handwritten or printed text into computerreadable text. It is the method by which printed text is digitised so that it can be accessed electronically, stored in a smaller space, accessible from the World Wide Web and is a field of research in the areas of pattern recognition, artificial intelligence and computer understanding of images.

Template matching involves the recognition of characters from ready-made templates or character outlines. The scanner digitises the image of a document on the computer and the Visual Recognition software tries to match, with a degree of probability, the characters from the scanned image file with the templates stored in it. If the image of a character corresponds to a recognised character, then it is matched with a text character for the computer. The Tesseract-OCR engine is considered one of the best Open Source machines in visual character recognition. The original versions could only recognise English, but now it can recognise text in more than 35 languages. One way in which it achieves this is through its built-in vocabulary, which allows it to correctly recognise words even if one of the characters would be misidentified as an individual [59] [60] [61].

Tesseract allows the user to customize some parameters, in order to improve the identification results. During the initialisation process of the application, a language

and its corresponding dictionary, i.e. the words it contains, are loaded. It is also possible to specify a subset of characters which will be the only ones accepted as recognition results, called a whitelist. In addition, it can be split so that at first it only recognizes letters and then only numbers, which can also improve recognition and is particularly relevant to this thesis [59].

Tesseract is suitable for use in the backend, but can also be used for more complex OCR tasks including layout analysis using a frontend interface. [62] [60]. Some of its applications focus on reading text from scanned images of printed documents, finding the text in complex images, or reading handwritten text.

Chapter 3

Methodology

This thesis introduces a driver system for autonomous vehicles, which was created utilizing Python programming language. This chapter outlines the development process of the system and the applied methodology.

3.1 Subsystems

Within the scope of this thesis, three distinct subsystems were developed: perception, navigation, and path planning. The perception subsystem attempts to extract and interpret valuable insights from images captured by the integrated camera, specifically related to road lanes, as well as static and dynamic obstacles in the environment.

In the navigation subsystem, a Lane Keeping system is implemented, using the lane detections accomplished by the perception subsystem. This mechanism strives to ensure the vehicle remains within the designated road boundaries. Additionally, another system is introduced, incorporating a PID controller and reliant solely on GPS navigation. This system monitors the vehicle's position and tracks predetermined waypoints along the path.

Lastly, a fundamental path construction methodology is implemented, furnishing a basis for navigation.

The systems are described in detail in the following paragraphs.

3.1.1 Perception subsystem

The perception system is responsible for recording and calculating the data of the vehicle's environment, i.e. all the information necessary for the autonomous vehicle to operate, such as information on the road structure, the positions of vehicles and pedestrians around it, traffic lights, the position of the vehicle in relation to the road lanes and the road boundaries. The subsystem is also responsible for interfacing the CARLA simulator with the rest of the autonomous vehicle system, i.e. it acts as a bridge between the autonomous navigation software created in this thesis and the CARLA simulator. The 'perception' system works by assuming that

all necessary information related to the road is received from the camera installed in the vehicle.

3.1.2 Navigation subsystem

The "navigation and control" subsystem is a low level subsystem used to control the movement of the vehicle. Given the desired path and the information received from the perception subsystem, the system generates the appropriate values for the vehicle's pedals (throttle, brake) and the steering wheel rotation angle to give the vehicle the desired speed and direction, and the resulting values are then applied to the vehicle via the CARLA simulator.

3.1.3 Path planning subsystem

The "path planning" subsystem uses the CARLA simulator map and a target point to provide the functional path to be followed by the vehicle. It is also responsible for locating the vehicle within the generated path.

3.2 Perception System

3.2.1 Introduction

The perception process in an autonomous vehicle is carried out using cameras to process and understand the environment around the vehicle in real time. The perception system of an autonomous vehicle observes the environment and its interaction with the vehicle. It is responsible for performing functions such as sensing, sensor fusion, localisation of objects in the environment and, in general, functions similar to the corresponding process of human visual perception. It is one of the most important subsystems of an autonomous vehicle as it is responsible for continuously collecting information such as traffic lanes, traffic signals, the position of dynamic and static objects in the surrounding environment, etc. By integrating all these tasks, the state of the outside world can be understood on the basis of sensor data. In addition, it should be able to function equally effectively under different weather conditions and throughout the day and night. Since the perception system also processes and generates information for the other subsystems of the vehicle, it is clear that a reliable and accurate perception system is extremely important.

3.2.2 Implementation

To obtain information from the environment related to objects and space, the RGB camera offered by the CARLA simulator was used. The information obtained is appropriately processed and converted into information for the autonomous vehicle. It is assumed that the camera is mounted on the roof of the vehicle and that the perspective it captures is similar to that of the driver.

The raw data is taken from the camera and then processed through CARLA's PythonAPI to extract the following information:

- The lanes on the roadway.

- The points of intersection.
- The position and value of the incoming speed limit signals.
- The status and position of the traffic lights.
- The position of dynamic obstacles, i.e. vehicles, bikes and pedestrians

The perception system developed within this thesis incorporates the necessary algorithmic procedures to transform the camera-acquired data into valuable insights about the vehicle.

3.2.3 Lane Detection

The lane detection on the road surface is part of the autonomous vehicle's perception system, and the information extracted from this process in relation to the position of the vehicle on the road should be used to take the corresponding actions in other subsystems. The algorithm developed has specific steps and the methodology is quite common for most problems of this type.

The typical procedure followed for pre-processing and lane detection is as follows [63]:

1. collection of the image
2. conversion of the image to grayscale
3. application of Gaussian filter for noise reduction
4. use of the Canny Edge Detector to identify areas of the image that have a sharp change in brightness
5. isolation of a region of interest in the image
6. Hough transformation
7. identification of the edges in the region of interest
8. grouping and final selection of the edges marking the left and right lanes
9. calculation of the coordinates of the start and end points of the edges that will represent the lanes in the image

We applied the algorithm to images collected from the simulator and then the system was integrated into CARLA. The lane detection process is a relatively simpler problem for ideal weather and lighting conditions, while its difficulty may increase in other conditions such as shade, fog, heavy rain, night landscape. For this reason, it is important to test the algorithm in many different conditions.

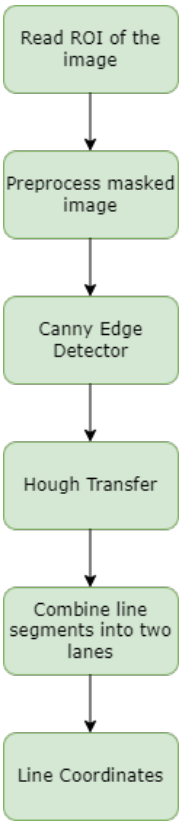


Figure 3.1: Lane Detection Algorithm

All the steps taken are described in detail below.

Conversion to grayscale

The color of a pixel in an image is created by a combination of three colors, red, green and blue (RGB). RGB color values are represented by a three dimensional vector (x, y, z). A color image is qualitatively dependent on the color represented by the number of bits. If each position for red, green and blue takes up 8 bits then the RGB combination takes up 24 bits and supports 2^{24} different colors.

The 24 bits represent the color of a pixel in the color image, while the corresponding grayscale image is represented by luminance using 8 bits. The luminance of a grayscale pixel value ranges from 0 to 255. A value of 0 indicates zero luminance, black color, while a value of 255 indicates maximum luminance, white color. The conversion of a color image to a grayscale image converts the RGB values, i.e. the conversion is from 24 bits to 8 bits.

It is important to understand that an image can be represented as a table, where each pixel of the image is a corresponding entry in the table. By calculating the gradient of the image, changes in brightness in a series of adjacent pixels can be found. A large gradient indicates a sharp change, while a small slope represents a small change. This helps us to identify edges in the image, since an edge is defined by the difference in brightness values. By converting a color image into a graycale image, as shown in Figure 3.2, we are able to minimise the computational processing time of the image, which is desirable in the real-time lane detection process.



Figure 3.2: Convertion to grayscale

Noise Reduction

Edge detection is prone to noise, so it is necessary to remove the noise from the image. This is achieved by applying filters to the original image.

In image processing, the Gaussian Filter or Gaussian Blur is used to reduce noise in the preprocessing stage of the image. The Gaussian Filter is a low-pass filter that uses a Gaussian function to calculate the transformation to be applied to each pixel of the image.

When the filter is applied, a change in the value of a pixel is made with the average value of the pixel brightness around it. The averaging of the pixels in the image to reduce the noise is done with the kernel, which takes values from the normal distribution.

In particular, the kernel of normally distributed numbers runs over the entire image and sets each pixel equal to the weighted average of the neighboring pixels, thus smoothing the image from noise. In the Figure 3.3 we use blurring in a gray image with a kernel size of 5×5 and a standard deviation of $\sigma = 0.0$.



Figure 3.3: Noise reduction with Gaussian filter

Canny Edge Detector

Canny Edge Detection is a popular edge detection algorithm. It was developed by John F. Canny in 1986. The image edge detection technique is a method that allows both the determination of the structural information of the objects contained in an image and dramatically reduces the amount of information that a system must process in order to complete certain processes, such as the detection of lanes. Figure 3.4 shows the result after applying the Canny Edge Detector to an image that has already been converted to grayscale and has undergone noise reduction.

The Canny Edge Detector algorithm has prevailed because of its performance criteria:

- **High detection rates:** The aim is to have a low probability of a point being falsely detected as an edge and at the same time a low probability of a true edge point not being identified as such.
- **Good localization:** Points marked edges by the operator should be as close as possible to the actual edges of the image.

Region of interest (ROI)

To increase processing speed, it is necessary to ensure that computing resources are not spent on unnecessary calculations. To achieve this, we need to select the area of the image that we are interested in processing, leaving the rest of the image unchanged. The Region of Interest (ROI) is a part of the image that is to be processed. In computer vision applications, the (ROI) is defined as the boundaries of the region containing the object to be examined. The OpenCV library provides a function for selecting regions of interest. However, there are alternative ways of selecting such



Figure 3.4: Canny Edge Detector

a region of interest, such as specifying a particular selection window shape, as was done in this thesis and can be observed in Figure 3.5. By creating a mask and restricting the region of interest to the roadway space in front of the driver, only the road lines in front of the vehicle are detected.

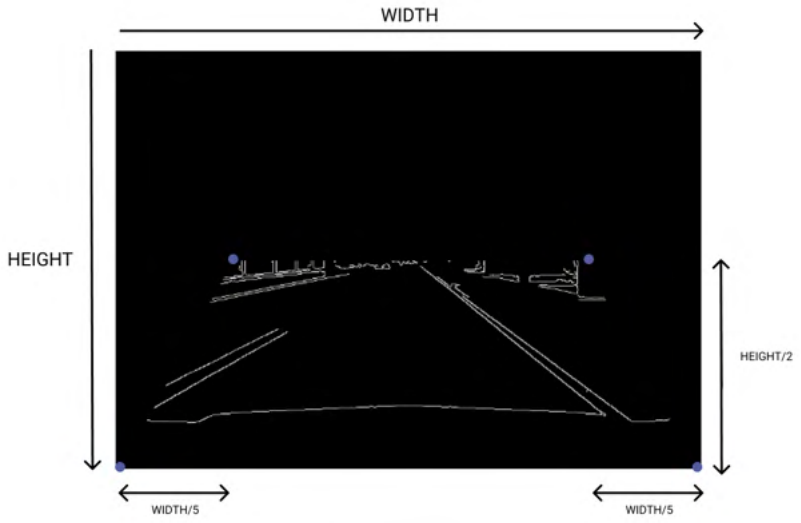


Figure 3.5: Region of Interest

Hough Line Transform and Lane Detection

After the preprocessing stage, the Hough Transform is applied to the image for final line detection. We have already discussed in detail the theoretical background of the Hough Transform and the Hough space in Chapter 2. The OpenCV

library provides us with a function for applying the transformation, which returns the parameters of the lines it finds in the region of interest.

The resulting lines are usually many and not all of them are representative of the lanes. For this reason, a filtering of these should be done. First, we group the edges into left and right lanes according to their slope and the parameters returned by the Hough Transform, while keeping in each list only those with slopes exceeding a threshold. This is important in order to avoid possible errors in detection, such as lines that tend to become horizontal and distort the result. A negative slope marks the left lane, while a positive slope marks the right lane. Next, we find the average for each list of edges, define the starting and ending points of each resulting edge, and project the resulting lines onto the original image, as shown in Figure 3.6.



Figure 3.6: Lane Detection on CARLA Simulator Image

3.2.4 Object Detection

Object Detection is one of the basic functions of the autonomous vehicle perception system. As already mentioned in Chapter 2, object detection requires object localisation and object classification in the image obtained by the simulator.

Of the algorithms that have been developed for this process, YOLO (You Only Look Once) was chosen. The problem has high demands on the speed of detection, so it was considered the optimal choice for the real-time object detection process, sacrificing small percentages of accuracy compared to the two-stage methods already reported. In this section, we discuss the process design and model training for object detection.

Dataset

For the implementation of the algorithm, a dataset of images collected from the RGB camera mounted on the vehicle in the CARLA simulator environment and from an existing data collection [64] was used. The dataset is publicly accessible to the reader by clicking here or by following the link: https://drive.google.com/drive/folders/1YeOABbyuXxyu1U1rZykbhRvpYQ23bFfF.

The dataset consists of 3624 images collected while the vehicle was driving in autopilot mode, which is provided by the simulator. It is important to mention that in this process it was desirable to collect images from different cities, maps and CARLA scenarios, with different weather conditions, as well as it is necessary that the road network contains all the dynamic obstacles that would be detected in the final process.

The classes defined in this thesis are 7: "vehicle", "bus", "bike", "motorbike", "pedestrian", "traffic light" and "traffic sign". In view of the conditions for a dataset to be processed by YOLO, a further processing stage is needed to have the necessary structure. Once the images have been extracted, a file containing all the objects in the image must be created, i.e. a corresponding labels file. The format of this file have to follow the YOLO standard and shall be of txt format, with its name refering to the corresponding image file. The content of the file is organised in lines, each of which contains the class number and then the coordinates of the object. The class number refers to the index of its position in the class table that we have defined. This process was done using the open source software labelImg [65], which offers us the possibility to manually annotate each image in various forms. Figure 3.7 shows some snapshots of the labelImg interface during the annotation process.

The dataset was divided into a training set, which contains 2345 images, and a testing set, which contains 450 images. The resolution of the images to be trained was 1600×1200.

Google Colab

In this thesis, we use the Interactive Python Programming Environment, Google Colaboratory (Google Colab). Google Colab, a product of Google Research, allows programmers to write and execute Python programming language codes through their browser. This environment is an excellent tool for tasks that require deep learning. It is essentially a specialised version of the Jupyter notebook, which does not require any installation. Google Colab is a largely open programming environment, providing computing resources such as GPUs and TPUs, which are graphics processing units that significantly reduce the time required for the development of the software. execution of a programme. In addition, Google Colab already has all the necessary libraries installed, which are available at any time. A special feature of the program is that it performs automatic storage in the Cloud, so there is no risk of corrupting the algorithm that has already been implemented. This work uses version 3.7.12 of the Python programming language, and also implements the Tesla K80 GPU graphics processing unit.

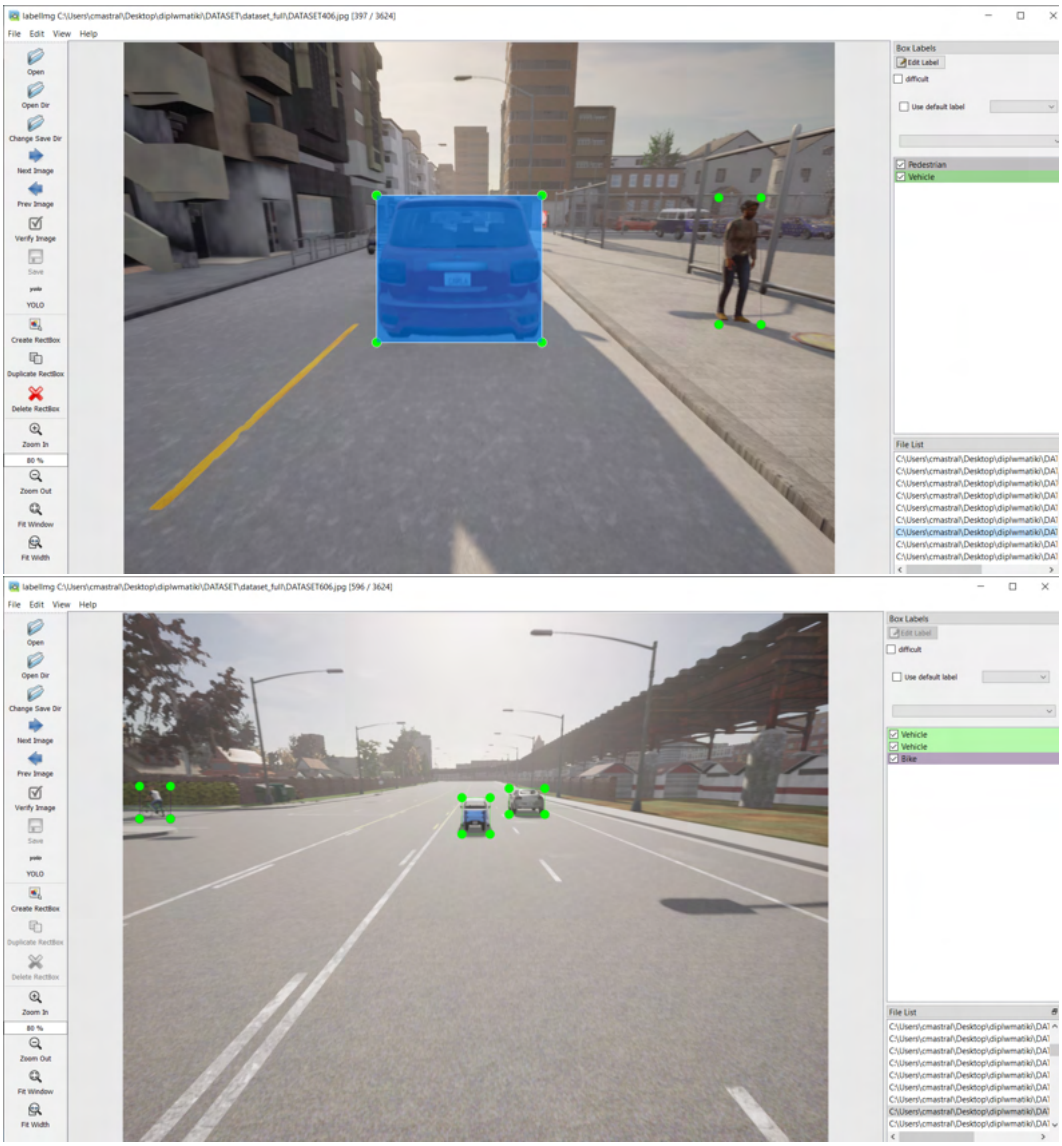


Figure 3.7: labelImg Enviroment

Training

As mentioned, the model used in this thesis is YOLOv4, the structure of which has been discussed in Chapter 2. First, we need to edit the corresponding **configuration file**, which is used to configure the parameters and initial network settings in order to process these parameters. The first step in the process is to change the **batch size** variable from 1 to 64. This variable denotes the number of samples (images) that will be passed through the network in each iteration, and this value has been found to be the optimal for both the memory required during the training process and its speed and accuracy. Also, the variable **max batches** is set according to the formula (classes x 2000), i.e. 14 000. This is because it has been shown that 2000 iterations for each class is approximately the number of iterations the network needs to perform during training. The **learning rate** will remain 0.001. The variable **steps** is defined according to the formula (80% of max batches, 90% of max batches), i.e. (11,200, 12,600). Also, the **network size** is set to be 416416 and the variable **classes** is changed to 7, which are the classes in each [yolo]-layer. Finally,

the number of **filters** is changed to 255 according to the formula filters=(classes + 5)x3 in each last convolutiona]-layer before each yolo]-layer.

Table 3.1: Model Parameters

Training Network Configuration			
batch	64	**learning rate**	0.001
subdivisions	16	**max batches**	14000
width	416	**steps**	11200, 12600
height	416	**classes**	7
channels	3	**flip**	0
decay	0.0005	**scales**	0.1, 0.1
angle	0	**burn in**	1000

Then, we create an **obj.names** file in the build/darknet/x64/data/ directory, with the class names each in a new order.

The **obj.data** file is created in the build/darknet/x64/data/ directory, which contains the following data:

The number of classes: classes = 7

The location for the train.txt file: train = data/train.txt

The location for the test.txt file: valid = data/test.txt

The location for the obj.names file: names = data/obj.names

The location of a backup folder so that all necessary files are stored there in case the training process ends or is abruptly interrupted: backup = backup/

The folder containing the dataset is placed in the directory build/darknet/x64/data/obj/ and contains, as already commented, the images (.jpg) and the corresponding labels (.txt) for each of the images. The file **train.txt** is created in build/darknet/x64/data/, which contains the names of the image files in the training set, for example:

data/obj/img1.jpg

data/obj/img2.jpg

data/obj/img3.jpg

We follow the same procedure for the files we have chosen to include in the test set, and for them we create the corresponding **test.txt** file.

Finally, we can change some parameters in the darknet, as we did for example in the case of this thesis where we decided to unpack the network weights (.weights file) every **100 iterations** instead of 1000 which was the default by the network.

3.2.5 Recognition of speed signals with Tesseract OCR

As already mentioned in the theoretical analysis, the use of the Tesseract OCR engine offers the possibility of reading text from a digital image. Its purpose in this work is to recognize the content written on the signals encountered by the vehicle in

the CARLA simulator environment. In particular, there are only signs for the vehicle speed limit (30, 60 and 90 km/h).

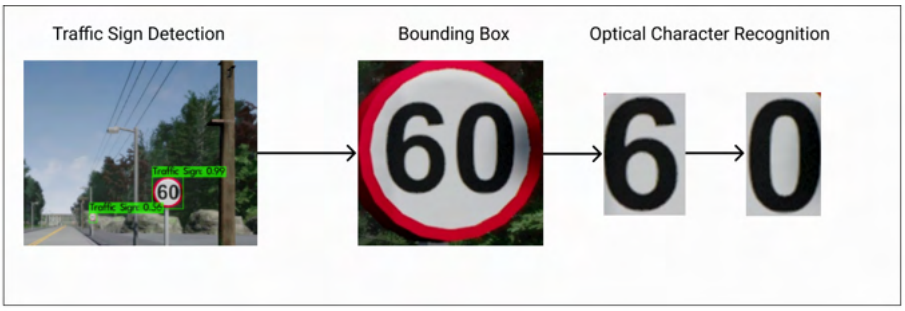


Figure 3.8: Speed signal recognition process

Since the offers different modes related to the structuring of the text in the input image (Page Segmentation Modes - PSMs) depending on the type of recognition the user is aiming for, in the thesis we used the type "9 - Treat the image as a single word in a circle". What is expected from input image, is to have a word in the frame of a circular shape, as the signals in the CARLA simulator look like those shown in Figure 3.8.

3.2.6 Histogram Analysis for color recognition

To identify the color of the traffic lights, the histogram information of the image can be used. The image histogram is one-dimensional for monochrome images and multidimensional for color images. The detection of peaks or valleys in the histogram can lead to object detection and entity separation respectively.

In colorful images, as in our case, the process is more complicated. The three histograms (RGB histograms) can be processed separately and the results combined. There is one histogram per channel R, G and B but also a histogram chart which is produced by averaging the three channels (R+G+B)/3.



Figure 3.9: Masks for detecting green and red

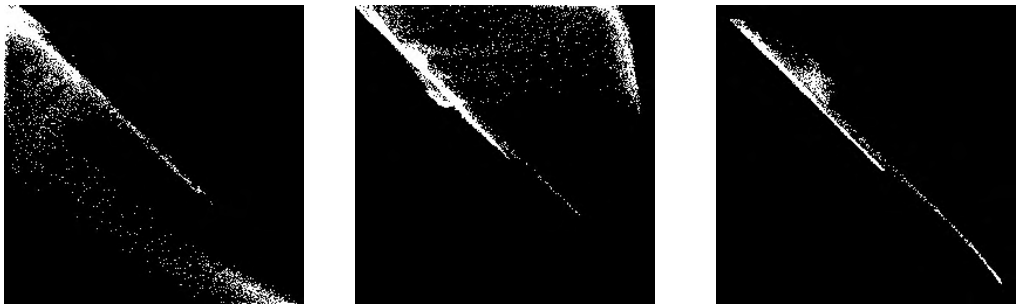


Figure 3.10: Histogram for green, red and orange respectively

Instead of considering the brightness distribution for the image overall, we can-consider each color channel separately. The procedure was derived from observing the combination of G and R in a 2D histogram. As already mentioned, with the help of 2D histograms we can analyse the channels (RG, GB, BR) in pairs or all together in a 3D histogram. In this particular case, what has interesting information is the GR histogram, as shown in Figure 3.10. Next, the values are adjusted/normalised to the scale (0,1) and two masks (Figure 3.9) are created to measure the corresponding points in the image that have high values in red and green respectively, while defining a threshold we can finally draw a conclusion about the color that appears in the image.

As shown in Figure 3.10, when the red color is present in the image, non-zero elements are shown in the upper and right part of the histogram, for green in the lower left part of the histogram and orange on the diagonal.

3.3 Navigation System

3.3.1 Navigation using PID Controllers

Over the years, and with the field of autonomous driving constantly evolving, many ways of controlling the vehicle have been proposed both by traditional and more modern methods, including machine learning techniques and neural networks. In this thesis we will examine some of the more traditional methods.

An efficient, widely used but low resource demand on the system is that of using PID Controllers, which will be incorporated into the CARLA simulator to control the vehicle in the road environment along a predefined path.

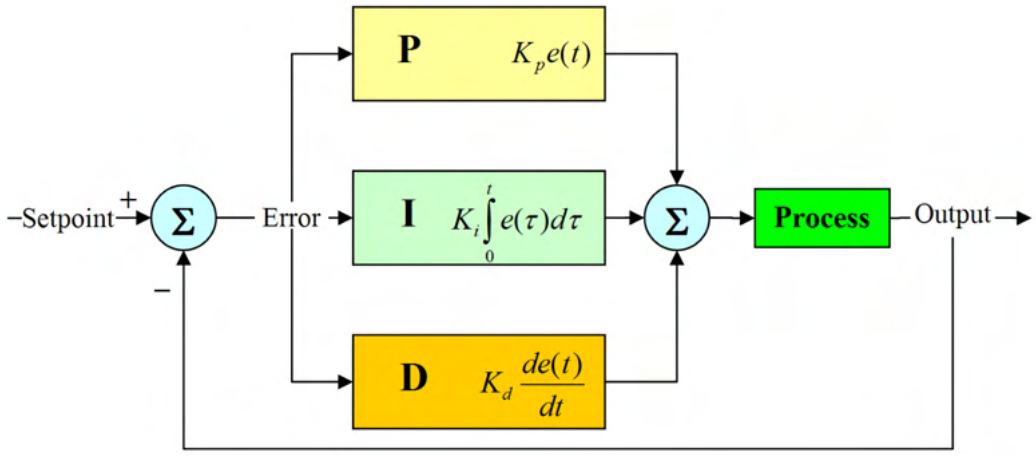


Figure 3.11: Block diagram of a PID controller [66]

A controller is simply an algorithm that takes some type of error signal and transforms it into an actuation signal to achieve a desired setpoint for a given process. The initials PID stand for three words: Proportional, Integral and Derivative and it is a control loop feedback mechanism. These controllers are widely used in the industry of control systems. A PID controller calculates an error value e(t) as the difference between a desired variable (setpoint) and a measured actual variable (process variable). Such PID controllers attempt to correct the actual variable to its desired one based on its proportional, integral, and derivative terms (denoted P, I, and D). These three terms are then summed to form the actual input to the main system [67], as can be observed in the controller diagram in Figure 3.11.

The terms K_P, K_I and K_D are respectively:

- K_P = Proportional gain
- K_I = Integral gain
- K_D = Derivative gain

For example, for a vehicle to stay inside the lanes of the road, the center of the lanes of the road would be the setpoint, i.e., the desired value, the tracking error would be the deviation from the desired position, the input would be the value to give the system to return the vehicle to the center of the lanes, i.e., to reduce the error, given the setpoint.

In order to understand the importance of the terms mentioned in the transfer function of the controller and the properties they offer, we will examine an example directly relevant to autonomous vehicles, that of speed control. In this case the controlled variable is the current speed and the setpoint is the desired speed, while the system is the vehicle physics and the engine.

Proportional term

The difference between the setpoint speed and the vehicles's current speed is called the error. When the vehicles's speed is below the setpoint, the error is positive, and when it is above the setpoint, the error is negative:

$$error_{speed} = setpoint_{speed} - current_{speed} \tag{3.1}$$

The proportional control term is the multiplication of this difference a scaling factor known as the proportional gain (K_P).

$$K_p * error_{speed} \tag{3.2}$$

So the value that should be given to the throttle of the vehicle would be:

$$Throttle = K_p * error_{speed} \tag{3.3}$$

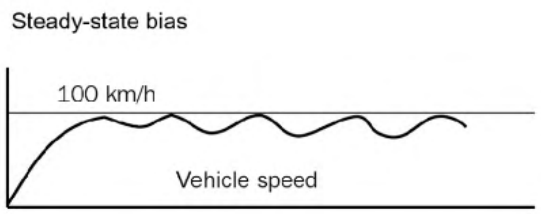


Figure 3.12: Steady state bias of the proportional gain [67]

In this case, as the speed error decreases, the throttle value decreases until zero error is reached and the throttle value is also zero. This means, however, that the system could never remain constant at the desired speed, as each time it reaches it, the throttle value would be reset to zero and directly the speed would fall back below the desired one, as shown in Figure 3.12. This means that the system would never really stay at the target speed and would rather oscillate just below our target speed, leaving us with a steady-state bias. Therefore, proportional control has the effect of decreasing the lift time, i.e., it makes the system faster, while it adjusts the permanent error and tends to zero it, but never does.

Integral Term

The integral term gives the solution to this challenge and seeks to address any steady-state bias in the system by integrating all the past errors in the system:

$$total_error_{speed} = total_error_{speed} + current_error_{speed} \tag{3.4}$$

The total error shall be calculated and scaled again by a factor, KI, called integral gain.

$$K_I * total_error_{speed} \tag{3.5}$$

$$Throttle = K_P * error_{speed} + K_I * total_error_{speed} \tag{3.6}$$

As explained, in the absence of this condition the speed would always be less than the desired speed so the overall error would be substantial. As time passes, the larger the term with the total error becomes. By choosing the appropriate KI the throttle value would remain greater than 0 even if the velocity error term is zeroed. Therefore, integral control increases the system type and zeroes the permanent error, but changes the transient behaviour for the worse as it causes oscillations around the desired value.

Derivative term

The last challenge can be addressed by the last term. The differential term is the solution to this as it adjusts the throttle value according to how fast the term approaches the setpoint. The rate of change of the error is defined:

$$K_D * (current_error_{speed} - previous_error_{speed})/(time_{current} - time_{previous}) \tag{3.7}$$

So we conclude:

$$K_D * error_{speed} * d/dt \tag{3.8}$$

As the error decreases, i.e. the value approaches the desired value, the term will be negative and will cause the value to decrease since we are now calculating the sum of all the terms explained.

$$Throttle = K_p * error_{speed} + K_I * total_error_{speed} + K_D * error_{speed} * d/dt \tag{3.9}$$

For the above to take place, tuning and making the right choice of gains is the key factor. The procedure presented was followed for the speed, braking and directional control of the vehicle by integrating the VehiclePIDController of the simulator into the system for the two controls, selecting the values of the gains and giving the desired speed. The desired path can be defined either by following the path that the simulator can generate or by following the ops created with the A* algorithm.

3.3.2 Lane Keeping

A navigation system designed to assist the vehicle to maintain its lane (Lane Keeping Assist System) includes two sub-processes: Lane Detection and Motion Planning. As already mentioned in the perception subsystem, the detection of the lanes on the roadway has been implemented using the OpenCV library and is able to extract from the images obtained by the camera installed all the necessary characteristics to obtain the coordinates of the straight lines which are the lanes on the roadway. The second procedure concerns the determination of the traffic. Since the system now has all the information about the position of the lanes, the next step is to determine the steer so that the vehicle remains within the boundaries of the lanes and ideally navigates in the middle of them. Therefore, given the lane information, the steering angle of the vehicle has to be calculated.

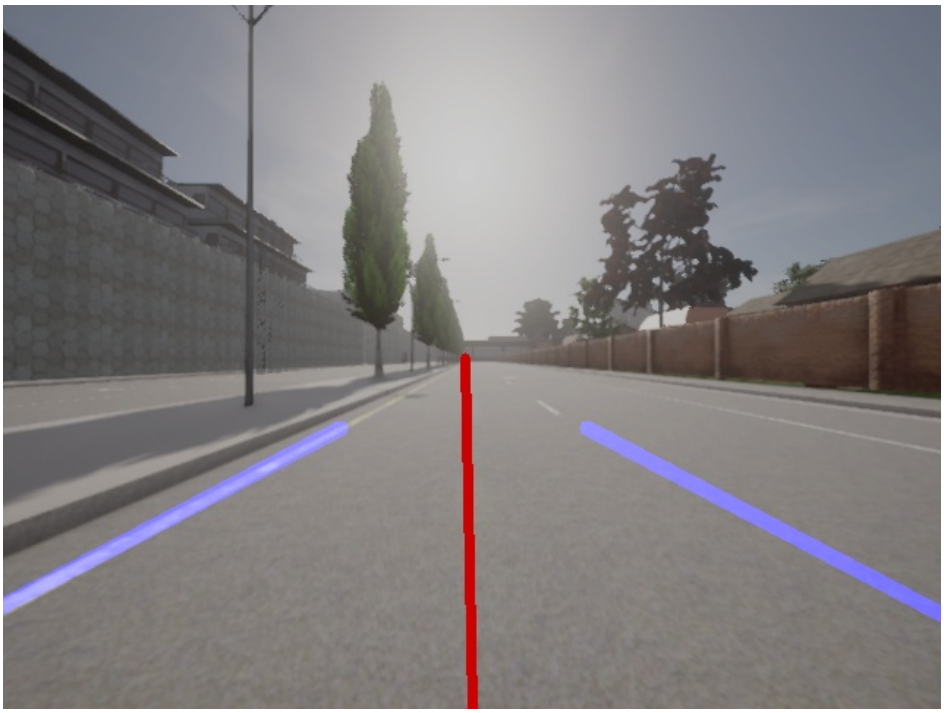


Figure 3.13: Heading line in the CARLA simulator environment

Heading Line

The first step in the procedure is to define the heading line. Since the vehicle must move in the middle of the lanes, the easiest way to do this is to first calculate the most distant endpoint of this line which will be located in the middle of the lanes that have been identified. Figure 3.13 shows the line with this point being the midpoint of the most distant points of the two lanes. Similarly, if the optics of the camera installed in the middle of the vehicle's field of view is known, the starting point is at the lowest point and again at the middle of the image.

Steering Angle

The last remaining procedure is to convert this information into a steering angle. As will be seen in Figure 3.14 below, the deviations on each axis must first be calculated. Since we assume that the camera is installed in the middle of the vehicle, we will calculate the deviation of the mean of the lanes from the mean of the image width and this will be x_{offset} , while y_{offset} is defined as height/2. As can be observed in the figure, the angle we want to calculate is ϕ.

$$\phi = \tan^{-1}(\frac{offset_x}{offset_y}) \tag{3.10}$$

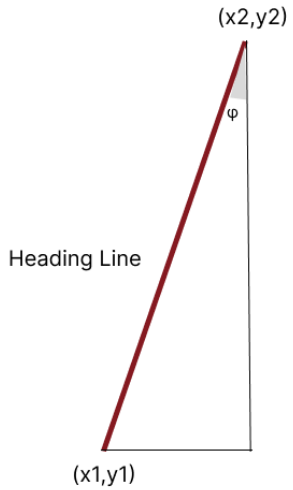


Figure 3.14: Calculation of the steering angle

Therefore, we can easily calculate the tangent of the angle, as shown in Figure 3.14, and arrive at its value, which is in rad. It can be converted to degrees and then to a value suitable for the vehicle in the CARLA simulator. The value of steer in the simulator can be set from -1 to 1, with negative values making the vehicle move to the left and positive values making the vehicle move to the right. These values correspond to -70 and 70 degrees respectively, but in different vehicles these values may change. On this basis, the steer value is calculated as follows:

$$steer = \varphi \cdot \frac{180}{70\pi} \tag{3.11}$$

Depending on the slope of the line, the appropriate sign will be given and the value will be checked to be between the range of values it should take. Finally, in order to avoid abrupt movements, a maximum value that the steer can take at any given time will be set, so that the vehicle can be progressively brought to the desired point without destabilising the vehicle. After this procedure, the steer value can be fed to the control.

3.4 Path planning

3.4.1 Introduction

The main use of an autonomous driving system is the transport from an initial point to a final destination. This process requires the design of a basic path, or route path, from the origin to the destination. This route must be the optimal one with respect to some criteria, mainly concerning the length of the path and the completion time of the route. In addition, the path to be constructed must respect the road code and not go outside the motorway limits, while at the same time driving in the correct direction according to the lanes, which is done by obtaining this road information from the CARLA API.

In this section, the design method and criteria for the design of the path are discussed, with the chosen design criterion being the length of the path. The algorithm A* has been used to find and design the path. A* is one of the best known algorithms in the field of robotics for path finding, its advantages include the running time and also that the solution is close to the optimal, while the most important disadvantage is that it does not find the optimal path. In our case the choice of A* is very good, using another algorithm that can provide the optimal solution such as the Dijkstra algorithm for example would make the runtime prohibitively long due to the complex structure of the dynamic space and the criteria that must be considered for the design.

3.4.2 Implementation

A* is one of the most well-known and widely used AI algorithms used for path search and graph traversal. It belongs to the category of informed or heuristic search algorithms. These algorithms use a heuristic function which helps them to evaluate the next possible states and select the worst of them. The aim of the algorithm is to find the optimal solution, i.e. the shortest path at least cost between two points called nodes.

The algorithm is widely used because of its accuracy and performance. It was originally presented in 1968 by Peter Hart, Nils Nilsson and Bertram Rapheal of the Stanford Research Institute. It is the result of combining greedy search with cost-based search.

It performs a classification of states (nodes) based on the function $f(k) = g(k) + h(k)$. The function g represents the cost of getting from the initial state to the current state, while the function h is a heuristic function that estimates the distance from the current state to the target state. The following figures show three examples of path design with A* in cities 1, 2 and 3 of the CARLA simulator. The algorithm steps in pseudocode form are presented below in Algorithm 1.



Figure 3.15: A* Implementation in Town01



Figure 3.16: A* Implementation in Town02



Figure 3.17: A* Implementation in Town03

Algorithm 1 Algorithm A*

```
1: Result: route
2: Node ← Class: Object with parameters: parent, position, g , h, f
3: # parent: Parent node of the current node
4: # position: Current node
5: # g: Distance value from starting position
6: # h: Distance value from ending position
7: # f: g + h
8: start: starting position
9: goal: ending position
10: open_list ← empty list
11: closed_list ← empty list
12: open_list ← append start
13: while open_list is not empty do
14:     current ← node with the lowest f score in the open list
15:     if this node is our destination node then
16:         Path Found
17:     else
18:         Put the current node in the closed list and look at all of its neighbors
19:         for each neighbor of the current node do
20:             if neighbor has < g value than current AND is in the closed list then
21:                 replace the neighbor with the new lower g value
22:                 current node is the neighbor's parent
23:             else if current g value is lower  this neighbor is in open list then
24:                 replace the neighbor with the new lower g
25:                 change the neighbor's parent to our current node
26:             else if this neighbor is not in both lists then
27:                 add it to the open list and set its g
28:             end if
29:         end for
30:     end if
31: end while
        return route
```

Chapter 4

Experiments and Results

4.1 Introduction

In this chapter the evaluation of the autonomous vehicle driving system developed during the thesis will be carried out. First, the assumptions made about the environment in which the system was tested and the metrics used to assess the quality and performance of the system will be presented. The evaluation criteria are based mainly on the accuracy of lane and obstacle detection, traffic sign information and traffic light color, as well as the measurement of the performance of the system in terms of the ability of the vehicle to navigate and complete its journey. The experiments are carried out in one of the cities of the CARLA simulator. Finally, the results of the experiments will be presented and the final conclusions will be drawn.

4.2 Assumptions

The vehicle operates in a road environment, the roads contain lanes and may be one-way or two-way. The roads contain speed limit signs and traffic lights. Dynamic environmental barriers are vehicles or pedestrians. The list of vehicles includes passenger vehicles, trucks, motorcycles and bicycles. The category of objects that the vehicle should be able to recognise includes all of the above, i.e. all vehicles, pedestrians, speed limit signs and traffic lights. Vehicles drive in accordance with the rules of the road but may break them. The autonomous vehicle is considered to receive data related to road information from a digital map. Static obstacles located off the road are not taken into account.

For the experiments, a computer with an Intel Core i7- 6500U processor with a processor frequency of 2.50GHz, 8 GB of RAM and an AMD Radeon R5 M335 4GB graphics card was used. The YOLO network was trained on Google Colab using the Tesla K80 GPU.

4.3 Evaluation Metrics

Before proceeding with the presentation of the experiments, we must briefly discuss how the models were evaluated. Following also [68], we define *tp, fn, fp, tn*

for a binary classification problem (the logic extends to multi-class problems such as ours), as shown in the table 4.1 We can now define (Table 4.2) in a summary table the common evaluation metrics.

Table 4.1: Definition of TP, TN, FP, FN

	Actual Positive Class	Actual Negative Class
Predicted Positive Class	True positive (tp)	False negative (fn)
Predicted Negative Class	False positive (fp)	True negative (tn)

Table 4.2: Definition of the metrics

Metrics	Formula	Definition
Accuracy (acc)	$\frac{tp+tn}{tp+fp+tn+fn}$	Measures the ratio of correct predictions to total predictions
Precision (p)	$\frac{tp}{tp+fp}$	Measure of how many of the positive predictions made are correct
Recall (r)	$\frac{tp}{tp+tn}$	Measure of how many of the positive cases the classifier correctly predicted, over all the positive cases in the data
F-Measure (FM)	$\frac{2*p*r}{p+r}$	It represents the harmonic mean between the values of recall and precision

The **IoU (Intersection over Union)** metric describes how the actual bounding box overlaps with the predicted bounding box and is a performance metric used in object detection. IoU evaluates the overlap by calculating the ratio of the overlap area to the combined area of prediction and ground truth. IoU is equal to 1 if the bounding box found is the same as the actual bounding box.

The metric used to evaluate the performance of the YOLO model is called the **mean average precision (mAP)**. Average precision (AP) is a popular metric for measuring the accuracy of object detectors such as YOLO. The average precision calculates the average precision value for the Recall value from 0 to 1.

The metric to be used to assess the **successful routes** is the completion rate averaged over N routes:

$$\frac{1}{N}\sum_{n=i}^{N} p_i$$

To find the **error of the lane detection** method, the actual edges representing the lanes and the edges from the application of the method were identified and the divergence between them was found by calculating the angle between the lines. To

find the acute angle ϕ formed by two lines ϵ_1 and ϵ_2 , first consider vectors $\vec{\delta_1} \parallel \epsilon_1$ and $\vec{\delta_2} \parallel \epsilon_2$ and find the angle $\omega = (\widehat{\vec{\delta_1}, \vec{\delta_2}})$ using:

$$cos(\widehat{\vec{\delta_1}, \vec{\delta_2}}) = \frac{\vec{\delta_1} \cdot \vec{\delta_2}}{|\vec{\delta_1}||\vec{\delta_2}|}$$

If $cos(\widehat{\vec{\delta_1}, \vec{\delta_2}}) > 0$, then $\omega < 90°$ and the angle is $\phi = \omega$.

Therefore, we can find the average error between the lines:

$$\frac{1}{N} \sum_{n=i}^{N} \omega_i$$

In addition, the standard deviation expressing the amount of variation or dispersion of the values can be obtained from the formula:

$$\sigma = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N} (x_i - \overline{x})^2}$$

4.4 Experiments and Results

4.4.1 CARLA Town Maps

Town01

Town01 is a relatively small city where all intersections are T-shaped, has few streets and does not contain difficult traffic structures. A small number of pedestrians and vehicles can be added to such a city.

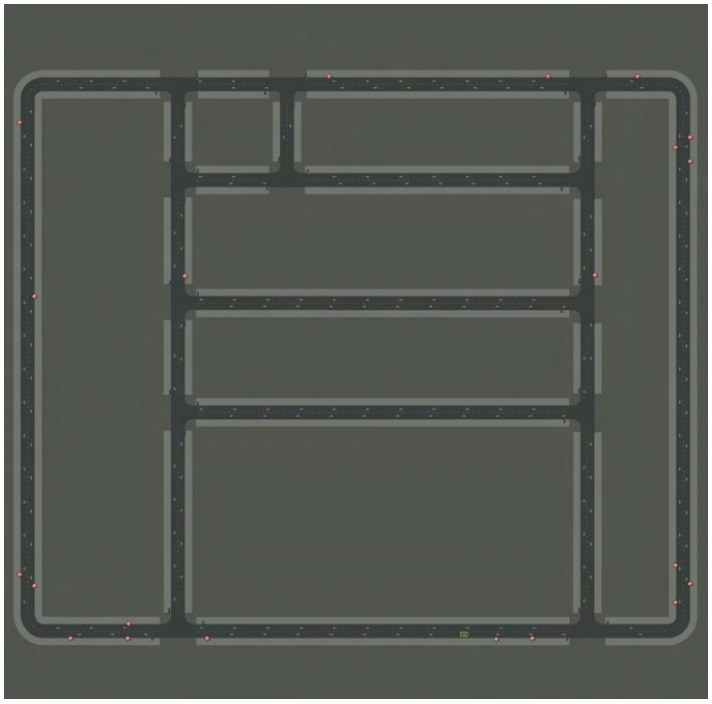


Figure 4.1: Town01 Map



Figure 4.2: Town01 Environment

Town02

Town02 is very similar to Town01, but is smaller in size. It has the same traffic environment as Town01 and can accommodate a small number of pedestrians and vehicles.

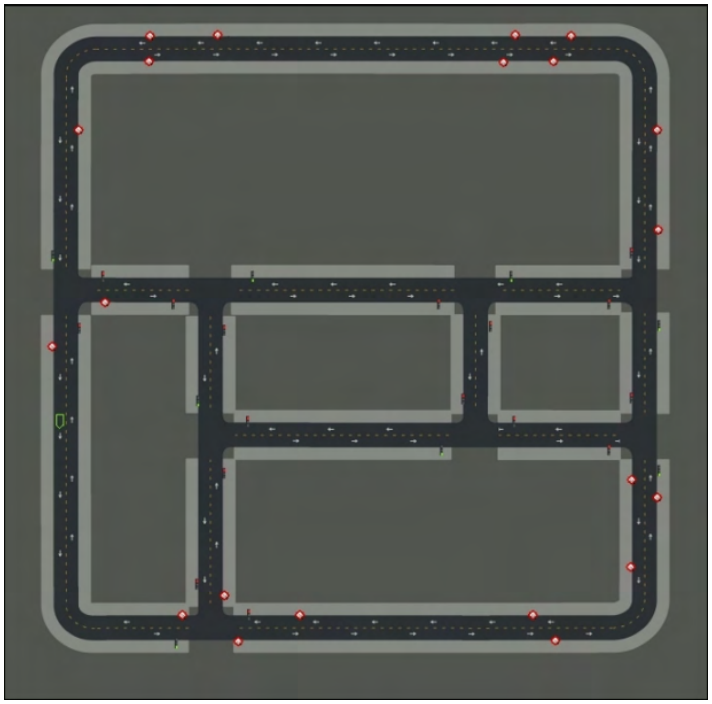


Figure 4.3: Town02 Map



Figure 4.4: Town02 Environment

Town03

This Town offers much more complex traffic scenarios as it has intersections with 4 lanes, and also contains a roundabout, traffic signals and side streets.

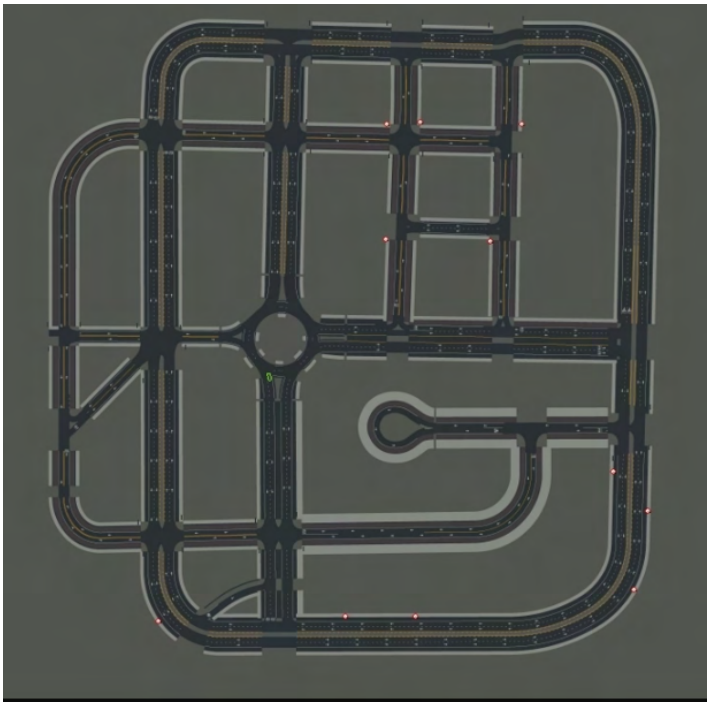


Figure 4.5: Town03 Map



Figure 4.6: Town03 Environment

Town04

Town04 supports a high volume of traffic as it has 4 lanes in each direction. In addition, it contains an exit from the motorway leading to a small town and an entrance to the motorway.

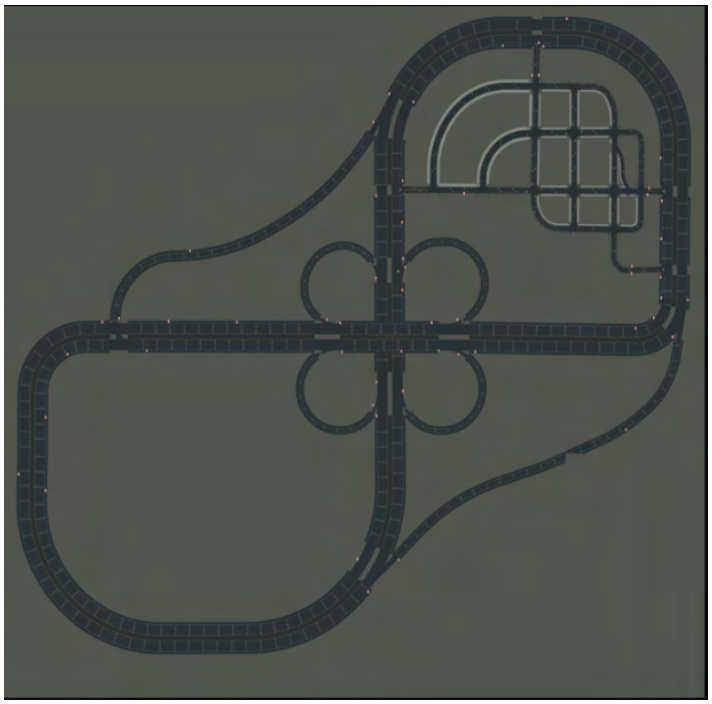


Figure 4.7: Town04 Map



Figure 4.8: Town04 Environment

Town05

It is a large town consisting of individual square blocks. It has a large bridge, one to three lanes of traffic and is ideal for small and large numbers of vehicles and pedestrians.

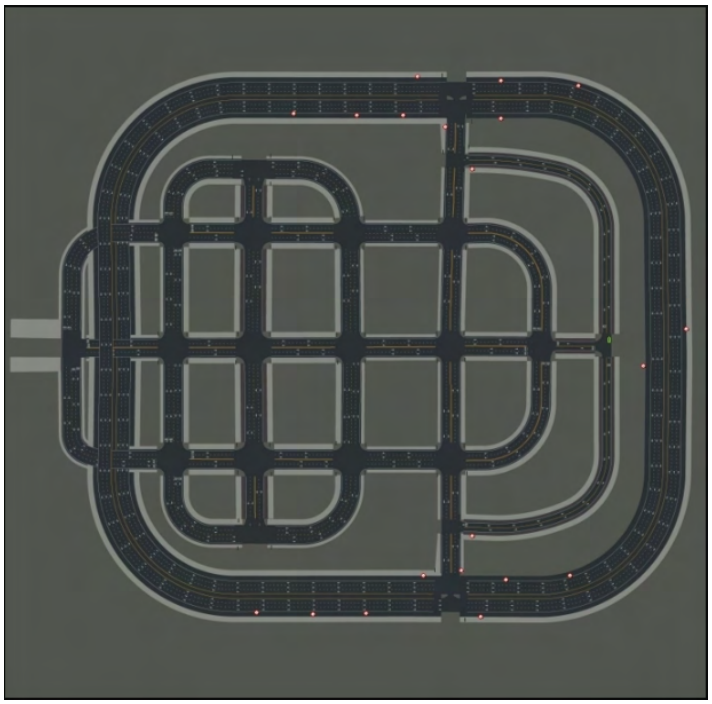


Figure 4.9: Town05 Map



Figure 4.10: Town05 Environment

4.4.2 Object Detection

As already mentioned in Chapter 3, a dataset of 3624 images from the CARLA environment was collected, and Tables 4.3 and 4.4 below contain the data for driving conditions and towns, as well as the set of images that contained objects to be identified (labeled) and the set that did not (unlabeled). Incorporating images without objects is important for enhancing the dataset's capacity to achieve robust generalization. Consequently, these images serve to enable the model to better adapt to diverse scenarios. Ultimately, the training set encompasses a total of 2345 images, whereas the testing set comprises 450 images.

Table 4.3: Images in training and test set

Dataset		
Town	Weather	Images
Town01	ClearNoon	422
Town02	ClearNoon	387
Town03	ClearNoon	301
Town04	ClearNoon	389
Town05	ClearNoon	386
Town01	CloudyNoon	300
Town02	CloudyNoon	59
Town03	WetCloudyNoon	115
Town04	SoftRainyNoon	132
Town05	CloudyNoon	183
Town01	SoftRainyNoon	121

Table 4.4: Labeled and Unlabeled images

Unlabeled	**Labeled**
755	1390

The evaluation encompassed the examination of metrics such as mAP (mean Average Precision), IoU (Intersection over Union), as well as TP (True Positives), FP (False Positives), and FN (False Negatives). These records from the model training runs were analyzed to find the optimal parameters, herein referred to as "best weights". These parameters, identified through this analysis, will ultimately be employed to generate the most accurate predictions.

Tables 4.5 and 4.6 below contain the results.

Upon analyzing the results, it becomes evident that the 11000 iterations the most exceptional performance within the dataset, making it the optimal choice for achieving the most accurate predictions. It's noteworthy that beyond this iteration, the performance begins to exhibit a gradual decline, a phenomenon attributed to overtraining.

For visual insight, Figure 4.11 showcases a selection of outcomes through images from the simulator environment.

Table 4.5: Model metrics during training

Metrics						
Weights Files	Precision	Recall	F1-Score	TP	FP	FN
1000 iterations	0.70	0.72	0.71	627	265	240
2000 iterations	0.81	0.78	0.78	680	120	187
3000 iterations	0.84	0.86	0.85	748	140	119
4000 iterations	0.89	0.84	0.87	731	90	136
5000 iterations	0.88	0.87	0.88	754	98	113
6000 iterations	0.89	0.88	0.88	761	97	106
7000 iterations	0.89	0.87	0.89	751	76	116
8000 iterations	0.89	0.89	0.89	770	98	97
9000 iterations	0.89	0.89	0.89	774	95	93
10000 iterations	0.88	0.88	0.88	766	100	101
11000 iterations	0.90	0.88	0.89	751	65	116
12000 iterations	0.89	0.89	0.89	768	87	99
13000 iterations	0.89	0.88	0.89	766	83	101
14000 iterations	0.88	0.88	0.89	768	84	99

Table 4.6: The mAP and IoU performance metrics during training

Weights Files	mAP	IoU
1000 iterations	63.83%	51.88%
2000 iterations	80.76%	68.61%
3000 iterations	84.97%	67.96%
4000 iterations	86.43%	73.17%
5000 iterations	87.40%	71.90%
6000 iterations	87.54%	72.68%
7000 iterations	87.02%	75.38%
8000 iterations	87.51%	73.03%
9000 iterations	88.21%	73.74%
10000 iterations	88.28%	73.29%
11000 iterations	88.99%	76.65%
12000 iterations	90.65%	76.15%
13000 iterations	90.28%	76.56%
14000 iterations	90.26%	76.51%

4.4.3 Optical Character Recognition (OCR)

The Tesseract OCR engine was employed to identify the information inscribed on the signals encountered by the vehicle within the CARLA simulator environment. As previously mentioned, these signals pertain exclusively to vehicle speed limits (30, 60, and 90 km/h).

While it is plausible to execute this signal recognition process directly within the Object Recognition framework by incorporating three additional classes corresponding to the speed limits (30, 60, and 90 km/h), a hierarchical strategy was adopted for problem-solving. In this approach, bounding boxes are initially detected, followed by



Figure 4.11: Object detection on images from CARLA Simulator

the isolation of relevant information for Optical Character Recognition. Opting for an alternative methodology would substantially escalate computational demands, as it would entail the identification of 10 classes.

For the purpose of evaluating this process, a dataset consisting of 100 images containing velocity signals was utilized. The accuracy of the predictions was subsequently computed in relation to these images, and the outcomes are presented in Table 4.7.

Upon inspecting Table 4.7, it becomes evident that while the algorithm achieves an overall high level of accuracy, it does encounter instances where accurate predictions are not achieved. Particularly noteworthy are scenarios involving challenging

Table 4.7: Performance of the OCR

30 km/h	37
60 km/h	39
90 km/h	24
Testing Set Accuracy	**0.76**

environmental conditions, such as heavy rain and fog, instances where the landscape is characterized by shadows, and situations where the detected signal is situated a considerable distance away from the vehicle. In these specific circumstances, the algorithm struggles to produce correct predictions, underscoring its limitations.

4.4.4 Traffic Light Color Recognition

The methodology for recognizing traffic light colors followed the algorithm detailed in Chapter 3. To evaluate the effectiveness of this approach, a set of 100 images was extracted from the dataset to calculate the accuracy of the predictions. Notably, the chosen image set encompasses a wide spectrum of weather and lighting conditions. The results of these assessments are summarized in Table 4.8.

Table 4.8: Performance of the color recognition method

Red Traffic Lights	27
Green Traffic Lights	55
Yellow Traffic Lights	18
Testing Set Accuracy	**0.96**

4.4.5 Lane Detection

The detection of the lanes on the roadway was carried out using the Hough Line Transform, using the methodology detailed in Chapter 3. The following figures show some of the tests carried out under different weather, road, and lighting conditions within the simulator environment. Notably, the method demonstrates commendable performance even in adverse conditions such as rain (depicted in Figure 4.13) or when encountering substantial shadows caused by obstacles on the lane (illustrated in Figure 4.14).

To assess the efficacy of the approach, a dataset of 50 images was chosen from instances characterized by typical daytime weather and lighting conditions—specifically, ClearNoon, CloudyNoon, and WetNoon. In these images, both the lines serving as the ground truth and those detected by the method were identified. The divergence between the two sets of edges was subsequently quantified.

By calculating the angle between the lines through the determination of the angle between two vectors parallel to the lines, the average divergence was determined to be approximately 1.06°, with a standard deviation of σ = 0.8.



Figure 4.12: Lane Detection in WetCloudyNoon conditions



Figure 4.13: Lane Detection in MidRainyNoon condition

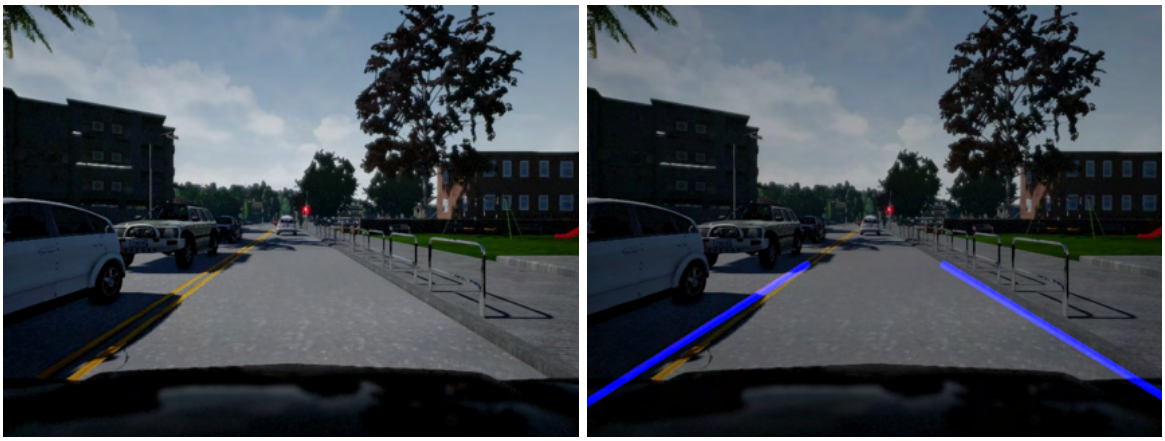


Figure 4.14: Lane Detection in ClearNoon conditions

Junctions

Within the CARLA simulator environment, road intersections are demarcated by a horizontal line on the road surface, as illustrated in Figure 4.15. The process for detecting these intersections parallels the procedure employed for lane detection, with the exception that a threshold is introduced for the slope of the detected straight lines. This threshold is configured to approximate the slope of horizontal lines,

specifically within the designated area of interest located ahead of the vehicle.

To assess the effectiveness of this approach, a dataset comprising 50 images depicting road junctions was utilized. The accuracy of the predictions was subsequently computed for these images, and the results are presented in Table 4.9.

Table 4.9: Performance of the junction detection method

Testing Accuracy	**0.70**



Figure 4.15: Junction detection

This method operates with a commendable level of accuracy; however, there are instances where the vehicle veers away from its intended position between the road lanes before reorienting itself. This particular scenario poses a challenge for the method's effectiveness. This limitation is not entirely surprising given the method's reliance solely on the camera input and its utilization of line slopes as a pivotal decision-making factor.

4.4.6 Navigation using PID Controllers

The navigation procedure, involving the integration of the VehiclePIDController from the simulator, was executed to manage the vehicle's speed, braking, and steering controls. The vehicle's position was tracked via GPS as it commenced its journey from the designated starting point, traversing a series of waypoints constituting the predefined path. These experiments were systematically conducted within CARLA Towns01, Town02, and Town03, each involving diverse execution routes and path lengths.

For each town, two distinct experiments were carried out, each following a unique route. The performance of the navigation on a specific route was assessed by

assigning a score of 0 in cases where the route could not be successfully completed, and a score of 1 if the vehicle achieved successful navigation along the specified route. The route execution success rates are meticulously documented in Table 4.10.

Table 4.10: Percentage of path execution with PID controller

Testing Accuracy	**0.83**

Within this methodology, an iterative experimentation process becomes crucial to determine the optimal values for the controller gains. These values are contingent upon factors such as waypoint distances and vehicle velocity. The refinement process continued until the suitable parameter values were successfully identified. However, it's noteworthy that one of these runs failed to complete, necessitating adjustments to certain parameters.

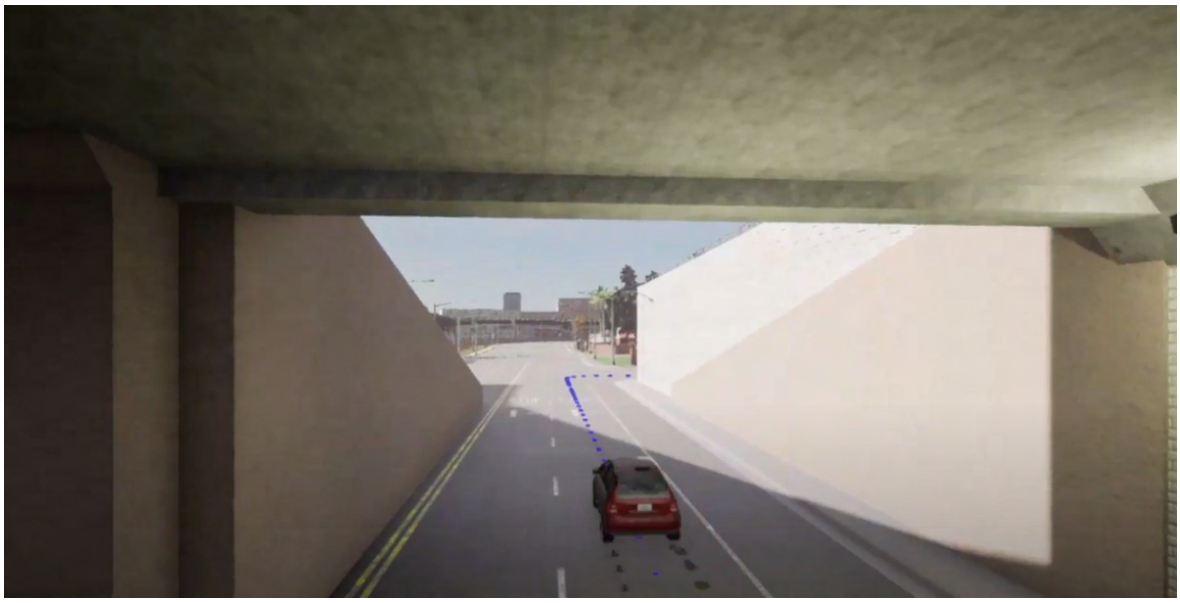


Figure 4.16: Navigation using waypoints

4.4.7 Lane Keeping

The Lane Keeping navigation process involves guiding the vehicle along a straight path, originating from a user-defined initial position, and achieving this without relying on GPS. The primary objective is to ensure that the vehicle remains within the lane boundaries solely by tracking and interpreting them, thereby issuing pertinent steering commands.

For the purpose of evaluating this methodology, routes devoid of intersections were selected. As discussed in the preceding section, Town01 and Town02 road networks devoid of intricate junctions, rendering them ideal for experimentation. Within each of these towns, a total of three routes were meticulously executed. The ensuing outcomes are documented in Table 4.11 below.

In this scenario, it became evident that the vehicle could adeptly navigate to its intended destination when confronted with clearly delineated road lanes. Nonethe-

Table 4.11: Percentage of path execution with the Lane Keeping system

Testing Accuracy	**0.67**

less, situations arose where minor irregularities in the road surface led to momentary discrepancies in the lanes detected by the system. Consequently, the vehicle occasionally executed abrupt maneuvers in its endeavor to reposition itself at the center of the lanes. Unfortunately, this behavior sometimes led to destabilization, preventing the vehicle from re-establishing its desired trajectory.

To address this issue, one potential solution is to employ an even higher threshold for the maximum steering control value achievable through the handlebars. Alternatively, a more effective approach might involve refining the method used to stabilize the steering angle, thereby enhancing the vehicle's capacity to respond smoothly and accurately to lane variations.

Chapter 5

Discussion and Conclusions

In this chapter, a concise summary of conclusions drawn from the experimental outcomes is presented, with suggestions for further system enhancement and development.

5.1 Conclusion

In the context of this thesis, three subsystems perception, navigation and path construction were implemented. The perception subsystem showcased the capability of extracting valuable insights from images and videos from the integrated camera and through Deep Learning we have managed to process images and effectively extract information with respect to static and dynamic obstacles in the environment. We were also able to interpret the information of road traffic signs with OCR, while digital image processing techniques were used to detect lanes and intersections on the road, and perform color recognition for traffic lights. Then, the navigation subsystem featured a lane-keeping system utilizing perception subsystem data, as well as a GPS-assisted PID controller for navigation and tracking, following the predetermined points of the path. Lastly, a path construction method for navigation was implemented.

On the basis of the results of the experiments in Chapter 4 concerning the ability of the vehicle to perform the individual functions mentioned, the following was observed:

1. The system effectively identified static and dynamic obstacles on the road network, performing exceptionally well for nearby obstacles. The system's efficiency diminished for distant obstacles, a limitation anticipated.
2. In the lane detection experiments, the method used was able to give fairly good results and to respond to different weather and lighting conditions. Intersection detection demonstrated similar behavior, yet the vehicle's position on the road surface significantly influenced the method.
3. Optical character recognition proved susceptible to weather and lighting conditions.

4. The lane keeping process demonstrated competent results, with room for refinement, particularly in areas where lanes are absent during turns on larger roads.

5.2 Challenges

One of the initial challenges encountered during the development of the thesis was the difficulty in finding suitable techniques to indicate how the methods presented in the literature, with the camera as the main sensor, should be combined to achieve an integrated behaviour. Although individual methods were discerned, there were not enough reports on complete architectures showing the integration of all subsystems.

Another problem encountered was the selection of the PID controller values for speed and position control of the system. The setting was initialized by testing the program in different scenarios. Iterative testing across various scenarios was necessary, with troubleshooting and resolution consuming additional time.

The biggest obstacle encountered during the work was the high computational resource requirements of CARLA. Experiments involving numerous vehicles and pedestrians, particularly at elevated speeds, resulted in the simulator's operation dropping below the optimal 5 fps threshold. More specifically, 8GB of RAM and a Radeon R5 M335 graphics card with 4GB of graphics memory were used. The simulator recommended using at least 470 GTX graphics with 8 GB of internal memory, so it was expected that there would be limitations during execution. Consequently, results are notably reliable for lower computational loads, such as image extraction and processing in smaller urban settings. More elaborate experiments involving navigation in larger, complex cityscapes necessitate additional computational resources.

5.3 Future Work

One improvement that could be made in this thesis is in the autonomous vehicle perception system. Utilizing the CARLA simulator's programming environment, data extraction from images was employed to construct a dataset for training the YOLO model. Expanding this dataset could increase model performance. However, the process is inherently time-intensive.

Furthermore, a navigation system reliant solely on visual cues could be implemented by studying more complex methods. Such a system would navigate and decide subsequent moves based on visual information from the camera's sensor. These methodologies often integrate neural networks with a state machine (FSM).This would incorporate an additional behaviour selection system to combine the individual subsystems.

Introducing analytical methods to establish evaluation rules in navigation assessment presents an additional improvement. A pre-defined table reflecting rule values for each behavior and their impact on evaluation metrics, coupled with a

rule selection mechanism for potential vehicle infractions, could enhance system evaluation.

In addition, for the evaluation of the lane detection method, an evaluation could be carried out with part of the dataset including more images with extreme weather and lighting conditions to provide a more accurate evaluation of the method's performance.

Lastly, envisioning an alternate route system for obstacle evasion or overtaking, driven by feedback from the perception system, presents a worthwhile prospect.

Bibliography

[1] NHTSA. *Preliminary Statement of Policy Concerning Automated Vehicle*. 2013. URL: http://www.nhtsa.gov/staticfiles/%20rulemaking/pdf/Automated_Vehicles_Policy.pdf.

[2] SAE International standard J3016 SAE International. *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*. 2014.

[3] VDA (German Association of the Automotive Industry). *Automated Driving*. 2016. URL: https://www.vda.de/de/themen/innovation-undtechnik/automatisiertes-fahren/automatisiertes-fahren.html.

[4] Tom M Gasser and Daniel Westhoff. "BASt-study: Definitions of automation and legal issues in Germany". In: *Proceedings of the 2012 road vehicle automation workshop*. Bergisch Gladbach: German Federal Highway Research Institute. 2012.

[5] Lloyd's. *Autonomous Vehicles: Handling Over Control: Opportunities and Risks for Insurance*. 2014. URL: https://assets.lloyds.com/assets/pdf-autonomous-vehicles/1/pdf-autonomous-vehicles.pdf.

[6] Tomtom. *What are the six levels of autonomous driving?* 2019. URL: https://www.tomtom.com/blog/automated-driving/different-levels-of-autonomous-driving.

[7] T. Litman. "Autonomous Vehicle Implementation Predictions". In: (June 2020). URL: https://www.vtpi.org/avip.pdf.

[8] *The Six Levels of Autonomous Testing Automated Visual Testing Applitools*. https://applitools.com/blog/not-only-cars-the-six-levels-of-autonomous/. Feb. 2019.

[9] Araz Taeihagh and Hazel Si Min Lim. "Governing autonomous vehicles: emerging responses for safety, liability, privacy, cybersecurity, and industry risks". In: *Transport Reviews* 39.1 (2019), pp. 103–128. DOI: 10.1080/01441647.2018.1494640. eprint: https://doi.org/10.1080/01441647.2018.1494640. URL: https://doi.org/10.1080/01441647.2018.1494640.

[10] Keshav Bimbraw. "Autonomous Cars: Past, Present and Future - A Review of the Developments in the Last Century, the Present Scenario and the Expected Future of Autonomous Vehicle Technology". In: *ICINCO 2015 - 12th International Conference on Informatics in Control, Automation and Robotics, Proceedings* 1 (Jan. 2015), pp. 191–198. DOI: 10.5220/0005540501910198.

[11] Ricardo A. Daziano, Mauricio Sarrias, and Benjamin Leard. "Are consumers willing to pay to let cars drive for them? Analyzing response to autonomous vehicles". In: *Transportation Research Part C: Emerging Technologies* 78 (2017), pp. 150–164. ISSN: 0968-090X. DOI: https://doi.org/10.1016/j.trc.

2017.03.003. URL: https://www.sciencedirect.com/science/article/pii/S0968090X17300682.

[12] *The Mercedes-Benz F 015 Luxury in Motion Forerunner of a mobility revolution*. https://group-media.mercedes-benz.com/marsMediaSite/en/instance/ko/The-Mercedes-Benz-F-015-Luxury-in-Motion-Forerunner-of-a-mobility-revolution.xhtml?oid=9906573.

[13] *Mercedes-Benz Innovation Autonomous*. https://www.mercedes-benz.com/en/innovation/autonomous/. Sept. 2020.

[14] Motortrend. *Tesla Model S*. https://www.motortrend.com/cars/tesla/model-s/2015/. Dec. 2015.

[15] Autoware. *Vision and Mission*. https://www.autoware.org/visionandmission. Aug. 2021.

[16] Shital Shah et al. *AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles*. 2017. DOI: 10.48550/ARXIV.1705.05065. URL: https://arxiv.org/abs/1705.05065.

[17] Alexey Dosovitskiy et al. "CARLA: An open urban driving simulator". In: *Conference on robot learning*. PMLR. 2017, pp. 1–16.

[18] *LG ELECTRONICS UNITY TECHNOLOGIES COLLABORATE ON AUTONOMOUS VEHICLE SIMULATION LG Singapore*. https://www.lg.com/sg/about-lg/press-and-media/lg-electronics-unity-technologies-collaborate-on-autonomous-vehicle-simulation. Apr. 2021.

[19] Jelena Kocić, Nenad Jovičić, and Vujo Drndarević. "Sensors and Sensor Fusion in Autonomous Vehicles". In: *2018 26th Telecommunications Forum (TELFOR)*. 2018, pp. 420–425. DOI: 10.1109/TELFOR.2018.8612054.

[20] Daniel Sales et al. "Vision-Based Autonomous Navigation System Using ANN and FSM Control". In: *Latin American Robotics Symposium and Intelligent Robotics Meeting* 0 (Oct. 2010), pp. 85–90. DOI: 10.1109/LARS.2010.26.

[21] Sebastian Thrun et al. "Stanley: The robot that won the DARPA Grand Challenge". In: *Journal of field Robotics* 23.9 (2006), pp. 661–692.

[22] Chris et al. Urmson. "Autonomous Driving in Urban Environments: Boss and the Urban Challenge". In: *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 1–59. ISBN: 978-3-642-03991-1. DOI: 10.1007/978-3-642-03991-1_1. URL: https://doi.org/10.1007/978-3-642-03991-1_1.

[23] Simon Herman and Khalid N. Ismail. "Single Camera Object Detection for Self-Driving Vehicle: A Review". In: *Journal of the Society of Automotive Engineers Malaysia* (2021).

[24] *How Autonomous Vehicles Sensors Fusion Helps Avoid Deaths*. https://intellias.com/sensor-fusion-autonomous-cars-helps-avoid-deaths-road/. Nov. 2018.

[25] A. L. Samuel. "Some Studies in Machine Learning Using the Game of Checkers". In: *IBM Journal of Research and Development* 3.3 (1959), pp. 210–229. DOI: 10.1147/rd.33.0210.

[26] Osvaldo Simeone. *A Brief Introduction to Machine Learning for Engineers*. 2017. DOI: 10.48550/ARXIV.1709.02840. URL: https://arxiv.org/abs/1709.02840.

[27] . 2021. URL: http://hdl.handle.net/10889/14454.

[28] Keiron O'Shea and Ryan Nash. *An Introduction to Convolutional Neural Networks*. 2015. DOI: 10.48550/ARXIV.1511.08458. URL: https://arxiv.org/abs/1511.08458.

[29] Sumit Saha. *A Comprehensive Guide to Convolutional Neural Networks*. https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53. Dec. 2018.

[30] Hamed Habibi Aghdam and Elnaz Jahani Heravi. "Guide to convolutional neural networks". In: *New York, NY: Springer* 10.978-973 (2017), p. 51.

[31] Dan C. Ciresan et al. "Flexible, High Performance Convolutional Neural Networks for Image Classification". In: *IJCAI*. 2011.

[32] G. E. Hinton and R. R. Salakhutdinov. "Reducing the Dimensionality of Data with Neural Networks". In: *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647. eprint: https://www.science.org/doi/pdf/10.1126/science.1127647. URL: https://www.science.org/doi/abs/10.1126/science.1127647.

[33] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proc. IEEE* 86 (1998), pp. 2278–2324.

[34] Olga Russakovsky et al. "ImageNet Large Scale Visual Recognition Challenge". In: *Int. J. Comput. Vision* 115.3 (Dec. 2015), pp. 211–252. ISSN: 0920-5691. DOI: 10.1007/s11263-015-0816-y. URL: https://doi.org/10.1007/s11263-015-0816-y.

[35] P. Viola and M. Jones. "Rapid object detection using a boosted cascade of simple features". In: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. Vol. 1. 2001, pp. I–I. DOI: 10.1109/CVPR.2001.990517.

[36] N. Dalal and B. Triggs. "Histograms of oriented gradients for human detection". In: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. Vol. 1. 2005, 886–893 vol. 1. DOI: 10.1109/CVPR.2005.177.

[37] Pedro Felzenszwalb, David Mcallester, and Deva Ramanan. "A Discriminatively Trained, Multiscale, Deformable Part Model". In: vol. 8: June 2008. DOI: 10.1109/CVPR.2008.4587597.

[38] Bernhard E. Boser, Isabelle M. Guyon, and Vladimir N. Vapnik. "A Training Algorithm for Optimal Margin Classifiers". In: *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*. COLT '92. Pittsburgh, Pennsylvania, USA: Association for Computing Machinery, 1992, pp. 144–152. ISBN: 089791497X. DOI: 10.1145/130385.130401. URL: https://doi.org/10.1145/130385.130401.

[39] Xin Yuan, Xiaosen Shan, and Li Su. "A Combined Pedestrian Detection Method Based on Haar-Like Features and HOG Features". In: *2011 3rd International Workshop on Intelligent Systems and Applications* (2011), pp. 1–4.

[40] *Using Histogram of Oriented Gradients HOG for Object Detection*. https://iq.opengenus.org/object-detection-with-histogram-of-oriented-gradients-hog/. Mar. 2020.

[41] M. Everingham et al. "The Pascal Visual Object Classes (VOC) Challenge". In: *International Journal of Computer Vision* 88.2 (June 2010), pp. 303–338.

[42] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848.

[43] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.

[44] Ross Girshick et al. *Rich feature hierarchies for accurate object detection and semantic segmentation*. 2013. DOI: 10.48550/ARXIV.1311.2524. URL: https://arxiv.org/abs/1311.2524.

[45] Georgia Gkioxari, Jitendra Malik, and Justin Johnson. *Mesh R-CNN*. 2019. DOI: 10.48550/ARXIV.1906.02739. URL: https://arxiv.org/abs/1906.02739.

[46] Ross Girshick. *Fast R-CNN*. 2015. DOI: 10.48550/ARXIV.1504.08083. URL: https://arxiv.org/abs/1504.08083.

[47] Shaoqing Ren et al. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2015. DOI: 10.48550/ARXIV.1506.01497. URL: https://arxiv.org/abs/1506.01497.

[48] Kaiming He et al. *Mask R-CNN*. 2017. DOI: 10.48550/ARXIV.1703.06870. URL: https://arxiv.org/abs/1703.06870.

[49] Joseph Redmon et al. “You only look once: Unified, real-time object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788.

[50] Wei Liu et al. “SSD: Single Shot MultiBox Detector”. In: *Computer Vision - ECCV 2016*. Ed. by Bastian Leibe et al. Cham: Springer International Publishing, 2016, pp. 21–37. ISBN: 978-3-319-46448-0.

[51] Jonathan Hui. *Object detection: speed and accuracy comparison*. https://jonathan-hui.medium.com/object-detection-speed-and-accuracy-comparison-faster-r-cnn-r-fcn-ssd-and-yolo-5425656ae359. Oct. 2020.

[52] *CARLA Simulator*. https://carla.readthedocs.io/en/latest/. Nov. 2021.

[53] *OpenCV*. https://opencv.org/. June 2002.

[54] Kari Pulli et al. “Realtime Computer Vision with OpenCV”. In: *Queue* 10 (2012), pp. 40–56.

[55] Linda G Shapiro, George C Stockman, et al. *Computer vision*. Vol. 3. Prentice Hall New Jersey, 2001.

[56] Gary Bradski and Adrian Kaehler. *Learning OpenCV: Computer vision with the OpenCV library*. " O’Reilly Media, Inc.", 2008.

[57] Byambaa Dorj and Deokjin Lee. “A Precise Lane Detection Algorithm Based on Top View Image Transformation and Least-Square Approaches”. In: *Journal of Sensors* 2016 (Nov. 2015), pp. 1–13. DOI: 10.1155/2016/4058093.

[58] *Intersection over Union IoU for object detection*. https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/. Nov. 2016.

[59] *Tesseract documentation Tesseract OCR*. https://tesseract-ocr.github.io/. May 2015.

[60] R. Smith. “An Overview of the Tesseract OCR Engine”. In: *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. Vol. 2. 2007, pp. 629–633. DOI: 10.1109/ICDAR.2007.4376991.

[61] Ray W. Smith. “History of the Tesseract OCR engine: what worked and what didn’t”. In: *Document Recognition and Retrieval XX*. Ed. by Richard Zanibbi and Bertrand Coüasnon. Vol. 8658. International Society for Optics and Photonics. SPIE, 2013, p. 865802. DOI: 10.1117/12.2010051. URL: https://doi.org/10.1117/12.2010051.

[62] Chirag I. Patel, Atul Patel, and Dr. Dharmendra Patel. “Optical Character Recognition by Open source OCR Tool Tesseract: A Case Study”. In: *International Journal of Computer Applications* 55 (2012), pp. 50–56.

[63] A Galip Ulsoy, Huei Peng, and Melih Çakmakci. *Automotive control systems*. Cambridge University Press, 2012.

[64] Martí Sánchez Juanola. “Speed traffic sign detection on the CARLA simulator using YOLO”. In: 2019. URL: http://hdl.handle.net/10230/42548.

[65] Heartex. “LabelImg”. In: *GitHub Repository* 6 (2015).

[66] Mattia Maldini. *PID control explained Medium*. https://maldus512.medium.com/pid-control-explained-45b671f10bc7.

[67] Krishtof Korda Luca Venturi. “Hands On Vision and Behavior for Self Driving Cars”. In: Oct. 2020. ISBN: 978-1-80020-358-7.

[68] Mohammad Hossin and Sulaiman M.N. “A Review on Evaluation Metrics for Data Classification Evaluations”. In: *International Journal of Data Mining Knowledge Management Process* 5 (Mar. 2015), pp. 01–11. DOI: 10.5121/ijdkp.2015.5201.